FollowThru — Continuation Prompt Playbook (15 prompts)

Fynd Hackathon 2026 · Challenge C14 · Chase what's late. Never what's done. · Starts from branch full-integration, commit cad765d · Single laptop, Claude Code in VS Code

1. How to use this playbook

This version compresses the earlier playbook's 26 copy-paste blocks (C01–C20 plus 6 operational prompts) into **15 prompts with the same functionality**. Every task, file, test, acceptance criterion and safety rule from the original is still here; related work now runs in one chat as numbered parts.

1. **One prompt per chat.** Open a new Claude Code chat, paste one whole grey block, and let it finish. Every block is self-contained.
2. **Run S01 first.** It writes `CLAUDE.md`, which every later session reads automatically.
3. **Go in order** (section 3). Essential prompts first; recommended and optional parts can be skipped without breaking anything.
4. **Two-part prompts** do Part 1, commit and push, then Part 2 in the same chat. If Part 1 fails its checks, Claude stops and reports before Part 2.
5. **Fixture-dependent prompt.** S02 needs the real challenge file at `data/fixture.json`. Until you have it, skip S02 and continue with S03. When it arrives, run S02, then re-run S04 Part 2 step 3 and S12.
6. **Do the "You check" list** after every prompt. If something is wrong, use S14 in FIX mode before moving on.
7. **After a usage limit or crash**, use S14 in RESUME mode with the last prompt id you ran.
8. **main is protected.** Claude merges into main only when you write exactly `merge into main`. Deploying, the APK and the GitHub release need your accounts (section 7).

Priority labels

Label	Meaning
Essential	Needed for the brief, the scoring evidence or a reliable demo.
Blocked	Waits for an input you must supply (the challenge fixture).
Recommended	Strong value for judges; safe to skip if time is short.
Optional	Enhancement from the original plan; cut first.

Rules every prompt enforces

Each block starts with the same project header: read the docs, inspect `git status`, keep uncommitted work, work only on `full-integration`, use the real scripts, keep the engine pure and the contract additive, stage files by explicit path, make small lowercase commits with no symbols,

no AI mentions and no co-author lines, never force-push or run destructive git commands, never merge into main without the exact phrase, and report only verified results.

2. Old prompts mapped to the new 15

New	Title	Replaces	Priority
S01	Project memory and blocker baseline	C01 + BLOCKER REPORT	Essential
S02	Challenge fixture intake and labelled accuracy	C02 + C03	Blocked until fixture
S03	Deadline parser and message signals	C04 + C05	Essential
S04	Reminder wording, escalation and product change record	C06 + C07	Essential
S05	Draft review actions and role-based navigation	C08 + C09	Essential
S06	Backup, evidence pack, audit filters and REST API	C10 + C11	Essential
S07	PWA completion	C12	Essential (needed for APK)
S08	Quality pass	C13	Essential
S09	Commitment detail explainability and landing story	C14 + C15	Recommended / Optional
S10	Optional AI-assisted extraction	C16	Optional (needs your key)
S11	Screenshot evidence and project documentation	C17	Essential
S12	Final QA and release check	C18 + RELEASE CHECK	Essential, re-runnable
S13	Merge, deploy on Vercel and Android APK	C19 + C20	Essential / needs your accounts
S14	Resume, fix or blocker report	CONTINUE + FIX + BLOCKER REPORT	Operational
S15	Scorecard and integration check	SCORECARD + INTEGRATION CHECK	Operational

Original plan tasks P01–P27 stay covered through the same mapping as before: P02/P04b → S01; P03/P04 adapter → S02; P05 → S03; P06/P07 → S03 (and S02 for real labels); P08/P11 → S04; P09/P10 → S04; P20/P12/P17/P21 → S05; P13/P22/P14 → S06; P24 → S07; P25 → S08; P19/P16 → S09; P23 → S10; P26 → S11; P27 → S12; deploy and APK → S13. P01, P15, P15b and P18 were already complete.

3. Order and dependencies

Order	Prompt	Needs first
1	S01	Nothing. Run first.
2	S02	S01 and `data/fixture.json`. Skip until you have the file.
3	S03	S01. Independent of the fixture. Re-run S02 Part 2 afterwards if S02 already ran.
4	S04	S03 (uses its not_done and progress events).
5	S05	S04 (subjects on cards, flag-aware approval).
6	S06	S05 (Part 2 also uses S04's suppressions CSV).
7	S07	S06 (vercel.json and API settled). Required before the APK in S13.
8	S08	S07. Runs after feature prompts so it checks final screens.
9	S09	S08 (lazy routes in place). Skip either part if short on time.
10	S10	S03 and S06. Skip if you have no key or no time.
11	S11	S08 (and S09 if you ran it, so screenshots show final screens).
12	S12	All essential prompts you chose to run (at least S01, S03–S08, S11).
13	S13	S12 with no unexplained FAIL, plus your phrases, Vercel URL and PWABuilder output.
any	S14, S15	Any time.

If the fixture arrives late: run S02, then re-run S04 Part 2 step 3 (starting state and final evidence) and S12. Parser and signal changes from S03 keep synthetic results unchanged, so the order is safe either way.

Verified starting point (17 Oct 2026, commit cad765d)

- Working branch `full-integration` at cad765d, identical on GitHub. GitHub `main` = 3fd93f3; `full-integration` contains all of main. `back-a-p0`, `back-a-p1`, `ui-b-p0`, `ui-b-p1` are fully merged into it.
- Scripts: dev, build, preview, test, test:watch, typecheck, evidence, e2e. Typecheck passes; 148 unit tests pass from `D:\followthru` (all fail from lowercase `d:/followthru`, fixed in S08);

build passes (main JS 658.69 kB, 209.19 kB gzip); e2e 4/4 pass.

- Challenge fixture absent; `CLAUDE.md` absent; only uncommitted file is your untracked `docs/TRACK_A_BACKEND_PROMPT.md`.
- External inputs: challenge fixture (S02), decision to commit it, Vercel account (S13), PWABuilder output and Android phone (S13), Gemini key (S10, optional), business case, pitch deck, demo video, teammate names, real growth-experiment rows, official deadline and demo format.

4. Continuation prompts

S01 · Project memory and blocker baseline

Essential · Replaces C01 + BLOCKER REPORT

Goal. Add the missing `CLAUDE.md` so every new session has the same rules, architecture map and commands, and record which external inputs are still missing.

```
PROMPT S01 - Project memory and blocker baseline

You are working on FollowThru (Fynd Hackathon 2026, challenge C14,
"Chase what's late. Never what's done.") in D:\followthru.
Single-laptop workflow: one developer, one branch, no Track A / Track B split.

BEFORE EDITING
1. Read README.md, CLAUDE.md (if it exists), docs/integration-status.md and
   docs/fixture-notes.md.
2. Run: git status --short ; git branch --show-current ; git log --oneline -8
3. Work only on branch full-integration. If you are elsewhere run
   git checkout full-integration and git pull origin full-integration.
   If git status shows changes you did not make in this session, STOP and
   show them. Never stash, reset or discard them. Leave the untracked file
   docs/TRACK_A_BACKEND_PROMPT.md untracked and untouched.
4. Run npm test once. If it fails before you change anything, report the
   failure and its output first.

PROJECT FACTS (verify them, do not assume)
- Stack: React 18, Vite 8, TypeScript strict, Tailwind v4, Dexie (IndexedDB),
  Zod 4, Vitest 5 + Testing Library + fake-indexeddb, Playwright (installed
  Chrome, channel "chrome"), vite-plugin-pwa, Vercel functions in api/.
- Scripts: dev, build, preview, test, test:watch, typecheck, evidence, e2e
  (plus any added by earlier prompts - check package.json).
- src/engine/types.ts is the single domain contract. No duplicate domain
  types elsewhere. Changes are additive (new optional fields, union members,
  types) unless I approve a breaking change. tests/contract.test.ts locks the
  public exports: update it on purpose and log every contract change in
  docs/integration-status.md.
- src/engine is pure and deterministic: never call Date.now() or new Date()
  there; the clock is EngineConfig.now. Entry runEngine(input, config) in
  src/engine/run.ts. Pipeline: signals.ts -> history.ts (resolveCommitments)
  -> drafts.ts (buildDrafts) -> approval.ts -> evaluate.ts. Flows in
  flows.ts, roles in permissions.ts.
- The UI reads data only through src/store (useEngine, useActions,
  useAuditLog, useEvidenceFiles, useRuns). Components never import Dexie.
  Every write goes through src/store/repo.ts and is audit-logged.
- Product rules: silence never changes status; only the owner, requester or
  a coordinator can close work; weak or ambiguous evidence sets needsReview
  and never changes status or triggers a chase; deadlines are read in the
  speaker's timezone and stored in UTC; one draft per commitment
  (id d_<commitmentId>) and updates bump its version; done, cancelled and
  deferred work keeps zero active drafts; approval re-checks freshness and
  needs a confirmed recipient; shadow mode only, nothing is ever sent.
- The challenge fixture is NOT in the repo unless data/fixture.json exists
  now. Never invent labels, fixture fields or accuracy numbers. Synthetic
  data (src/mocks/demoDataset.json, tests/fixtures/*.json) is always
  labelled synthetic and never presented as challenge evidence.
```

```
GIT AND REPORTING RULES
- Small focused commits. Before every commit run npm test, npm run typecheck
  and npm run build; all must pass. Review git diff --staged first.
- Stage files by explicit path. Never git add -A or git add .
- Commit messages: all lowercase, 3 to 8 words, plain English, no symbols or
  punctuation, no prefixes like feat or fix, one line, git commit -m "msg".
- Never add Co-Authored-By, "Generated with", or any mention of AI or Claude
  in commits, code, docs or PR text. Never change git config; never --author.
- Never commit node_modules, dist, .env files, API keys, keystores, APK files
  or personal data.
- Never force push, git reset --hard, git clean, git checkout -- ., rebase a
  pushed branch or delete branches. Never merge into main unless I write
  exactly: merge into main.
- After each commit: git push origin full-integration
- Report honestly. Never claim a test, build, deploy, score or screenshot you
  did not produce. Never delete, skip or weaken a test to go green.

PART 1 - CLAUDE.md (documentation only, no code changes)
1. Read C:\Users\shiva\projects\followthru\CLAUDE.md for its wording on
   product rules and design language. Do NOT copy its two-laptop section,
   its old folder list (src/state, src/lib) or anything not true today.
2. Write CLAUDE.md in the repo root with short factual sections:
   a. What we are building: coordinator and member roles, demo profiles,
      shadow mode, tagline.
   b. Scoring from the brief: 10 points owner/deadline/status for ten
      labelled commitments with evidence; 10 points zero reminders for
      completed, cancelled or deferred work; 5 points one action one draft
      and deadline updates change the same draft. Edge cases: maybe Friday,
      timezones, delegated owner, completion in another thread, no response
      versus not completed. Product change: stale-message suppression and
      recipient review; keep starting state, negotiated scope, final
      evidence.
   c. The 8 golden engine rules: silence never changes status; only owner,
      requester, coordinator or an authorised human decision changes status;
      ambiguous evidence -> needsReview, never a chase; now is always passed
      in; UTC storage, phrases read in the speaker timezone; every decision
      keeps evidence (message id, thread id, time, quote, rule); never
      hardcode fixture labels in the engine; one versioned draft per
      commitment, id d_<commitmentId>.
   d. Single-laptop workflow: branch full-integration, push to origin
      full-integration, main only after the exact phrase "merge into main",
      no force push, no destructive git commands.
   e. Architecture map with one line per real folder or key file
      (src/engine/*.ts, src/schema/*.ts, src/store/*.ts(x), src/pages/*.tsx,
      src/components/{ui,shell,domain}, src/three, src/styles,
      src/mocks/demoDataset.json, api/_lib/handlers.ts, api/health.ts,
      api/process.ts, scripts/evidence.ts, tests/, e2e/, vite.config.ts with
      the dev API plugin and PWA, vercel.json). Confirm every path exists
      with ls or Glob before listing it.
   f. Commands copied exactly from package.json, one line each.
   g. Data: fixture goes to data/fixture.json (say whether it exists now),
      canonical format in docs/fixture-notes.md, demo data is synthetic and
      never scored, demo option shows in dev or when VITE_DEMO_MODE=true.
   h. Contract rules for src/engine/types.ts and tests/contract.test.ts.
   i. Code rules: small files, no dead code, no console.log, tests for every
      changed business rule, test + typecheck + build before commits.
   j. Commit style: lowercase, 3-8 words, no symbols, no AI mention, no
      co-author trailers.
   k. Honesty rules: never invent results, URLs, scores or labels.
```

```
3. Keep it under about 160 lines. Its text must not mention AI tools.
4. Do not edit C:\Users\shiva\.claude\CLAUDE.md (my user-level file).
5. Check every listed path exists (a small loop), run test, typecheck and
   build, then commit: add project memory for single laptop

PART 2 - BLOCKER BASELINE (report only, no invented inputs)
6. Check and report each external dependency:
   - data/fixture.json present? committed?
   - official submission deadline and demo format confirmed?
   - Vercel project and live URL (look for LIVE_URL placeholders)?
   - PWABuilder output, assetlinks.json, APK release (APK_URL)?
   - optional Gemini key: never ask me to paste it into the chat;
   - business case PDF, pitch deck, demo video (DEMO_VIDEO_URL), teammate
     names (TEAMMATE_1, TEAMMATE_2), real growth-experiment rows?
7. For each missing item: what it blocks, who provides it (me or the
   organisers) and the exact action needed.
8. List the playbook prompts S02 to S13 that can run without these inputs,
   in dependency order, and name the next one to run.

ACCEPTANCE CRITERIA
- CLAUDE.md exists in the repo root, is pushed, every path exists and every
  command matches package.json; no two-laptop text; no AI tool mention.
- npm test, typecheck and build still pass.
- Blocker table delivered with owner and action for each missing input.

FINAL REPORT
Files changed; real results of every command (test counts, typecheck, build
sizes, e2e); commits with hashes; output of
git log -3 --format="%an <%ae>%n%b" to prove no co-author lines; the blocker
table; anything not done and why. Then stop and wait.
```

You check

- Open `CLAUDE.md` on GitHub (branch `full-integration`) and read it once. It must say work happens on `full-integration` and main needs the exact phrase.
- Start a new Claude Code chat and ask “what branch do we work on?” It should answer from `CLAUDE.md`.
- Read the blocker table and note what you must supply.

S02 · Challenge fixture intake and labelled accuracy

Blocked until you provide the fixture · Replaces C02 + C03

Goal. Load the real Fynd fixture, document every field and edge case with message ids, make it pass the validator (through an adapter only if needed), then reach a correct owner, deadline and status for every labelled commitment by fixing general rules — never hardcoded answers.

Before you run it: copy Fynd's `Starter_Data/C14/fixture.json` to `D:\followthru\data\fixture.json`.

```
PROMPT S02 - Challenge fixture intake and labelled accuracy

You are working on FollowThru (Fynd Hackathon 2026, challenge C14,
"Chase what's late. Never what's done.") in D:\followthru.
Single-laptop workflow: one developer, one branch, no Track A / Track B split.

BEFORE EDITING
1. Read CLAUDE.md, README.md, docs/integration-status.md and
   docs/fixture-notes.md.
2. Run: git status --short ; git branch --show-current ; git log --oneline -8
3. Work only on branch full-integration (git checkout full-integration ;
   git pull origin full-integration). If git status shows changes you did
   not make in this session, STOP and show them. Never stash, reset or
   discard. Leave docs/TRACK_A_BACKEND_PROMPT.md untracked and untouched.
4. Run npm test once. If it fails before you change anything, report first.

PROJECT FACTS AND RULES
All facts and rules in CLAUDE.md apply. In short: src/engine/types.ts is the
single contract (additive changes only, tests/contract.test.ts updated on
purpose, changes logged in docs/integration-status.md); src/engine is pure
(no Date.now, clock = EngineConfig.now); UI uses src/store hooks only; every
write is audit-logged via src/store/repo.ts; silence never changes status;
ambiguous evidence -> needsReview; one versioned draft per commitment; zero
active drafts on done, cancelled or deferred work; shadow mode only; never
invent labels, fields or accuracy; synthetic data is always labelled.

GIT AND REPORTING RULES
- test + typecheck + build pass before every commit; review
  git diff --staged; stage by explicit path, never git add -A or git add .
- Commits: lowercase, 3-8 words, no symbols, no prefixes, one line. No
  Co-Authored-By and no AI or Claude mention anywhere. Never change git
  config or use --author.
- No node_modules, dist, .env, keys, keystores, APKs or personal data.
- No force push, reset --hard, clean, checkout -- ., rebase of pushed
  branches or branch deletion. Never merge into main without the exact
  phrase: merge into main. Push after each commit:
  git push origin full-integration
- Never claim results you did not produce; never weaken tests.

CURRENT CONTEXT (verified at cad765d; re-check against the code)
data/ has only README.md. docs/fixture-notes.md is a template with a
canonical input format section. Validation: src/schema/validate.ts
(validateDataset, validateDatasetJson). Setup page bundles data/fixture.json
via import.meta.glob when present; scripts/evidence.ts reads it by default;
api/_lib/handlers.ts validates POST bodies with the same validator. The
engine has only been verified on synthetic data. Design choices to check
against real labels: a request to one person opens a commitment before they
```

```
reply (response pending); date-only deadlines -> 23:59 speaker time; EOD ->
18:00; "next Friday" = first Friday strictly after today; cross-thread links
use STRONG_LINK 0.5 and WEAK_LINK 0.3 in history.ts; evaluate.ts compares
deadlines within 60 seconds.

STEP 0. If data/fixture.json does not exist, STOP. Reply with one line:
"BLOCKED: data/fixture.json missing" and change nothing.

PART 1 - FIXTURE INTAKE (no engine rule changes in this part)
1. Never modify data/fixture.json. Ask me whether it may be committed (a
   deployed build needs it to offer "Load the C14 challenge fixture").
   Until I answer yes, do not stage it. Continue with the other steps.
2. Read the whole file (every record) and rewrite docs/fixture-notes.md.
   Keep "Canonical input format" and replace the template sections with:
   a. top-level structure with counts;
   b. a table of every field: path, type, required, example, meaning;
   c. people with ids, names, timezones, roles or teams;
   d. threads with ids, titles, participants, message counts;
   e. how timestamps and timezones are written, and every message whose
      local date differs from its UTC date;
   f. whether commitments, due dates, changes and completions are
      structured fields, free text or both;
   g. where labelled answers live, their exact format and status
      vocabulary, and a table listing every labelled item;
   h. the evaluation clock (explicit, or a proposal such as the latest
      message time, with the reason);
   i. an edge-case catalogue with message id, thread id and a short quote
      for: tentative deadline, timezone difference, delegation, completion
      in another thread, deadline changed later, cancelled, deferred or on
      hold, silence after deadline, explicit not done yet, anything
      ambiguous or messy;
   j. data quality problems.
3. Validate with a tiny tsx one-off (not committed) that runs
   validateDatasetJson on the file and prints the issues.
   - Zero errors -> no adapter.
   - Otherwise create src/schema/adapter.ts with pure
     adaptFixture(raw: unknown): unknown mapping the real shape onto the
     canonical DatasetInput (people, threads, messages, labels,
     referenceTime); return input unchanged when it already validates.
     Map the label status vocabulary to open, done, cancelled or deferred
     with a documented table; an unknown status is a validation error,
     never a guess; change label values only by that mapping. Document the
     field mapping in fixture-notes. Wire it in exactly three places: the
     loader in src/pages/Setup.tsx (before validateDatasetJson),
     scripts/evidence.ts (before validation) and handleProcess in
     api/_lib/handlers.ts (before validateDataset; accept body key
     "fixture" as an alias of "dataset"). Export it from src/schema/index.ts.
4. tests/adapter.test.ts: a small excerpt with the REAL field names (two or
   three records copied from the fixture) checking the mapping, plus a test
   that reads data/fixture.json and expects zero validation errors, using
   it.skipIf when the file is absent.
5. Run npm run evidence and paste its full scorecard summary.
   Commits: add real fixture notes / add fixture adapter (only if needed) /
   validate challenge fixture in tests

PART 2 - MAKE THE ENGINE PASS THE LABELS
6. Open the newest evidence/*/scorecard.md. For every failing labelled
   item inspect the commitment's events (rule names, evidence message ids)
   and classify the cause: signal detection (signals.ts), addressee
   resolution or linking (history.ts), deadline parsing (deadline.ts),
```

```
   title keywords for linking (titleFrom, keywords), authority rules,
   responseState, evaluation tolerance, or the adapter label mapping.
7. Print a diagnosis table BEFORE changing code: label id, expected,
   actual, cause, proposed general fix, risk to other cases.
8. Fix one cause at a time:
   - Never special-case message ids, people ids or label values in
     src/engine. After all fixes grep src/engine for every labelled message
     id: zero matches.
   - One regression test per fix. If the fixture is committed tests may read
     real messages; otherwise build a minimal synthetic excerpt with the same
     wording pattern, labelled synthetic.
   - After each fix run npm test and npm run evidence. Existing synthetic
     expectations in tests/engine.test.ts keep passing; if one must change,
     explain exactly why in the report.
   - New fields go into src/engine/types.ts additively; update
     tests/contract.test.ts only if exports change.
9. Stop when all checks pass, or when a remaining failure needs a human
   decision (for example a label that contradicts the brief). Report those
   with evidence; do not force them.
10. Keep the final passing evidence folder (script-generated, never hand
    edited) and commit it as challenge evidence. Run npm run e2e once.
    Commits: one per fix, for example: link completion across threads by
    owner / read deadline phrase in owner timezone / save passing challenge
    evidence

ACCEPTANCE CRITERIA
- data/fixture.json validates with zero errors (directly or via adapter).
- fixture-notes lists every labelled commitment with message ids and the
  full edge-case catalogue.
- Scorecard: all labelled commitments correct (or every remaining failure
  documented and agreed with me), 0 active reminders on closed work, no
  duplicate drafts.
- grep of labelled message ids in src/engine returns nothing; a regression
  test exists for every fix; all suites pass.

FINAL REPORT
Files changed; real results of every command (test counts, typecheck, build
sizes, evidence summary, e2e); the diagnosis table; commits with hashes;
git log -5 --format="%an <%ae>%n%b" output; my pending decision about
committing the fixture; anything not done and why. Then stop and wait.
```

You check

- `docs/fixture-notes.md`: labelled items and edge cases cite real message ids.
- `npm run dev`, open `http://localhost:5173/setup`, load the challenge fixture: zero errors.
- Compare the committed evidence `scorecard.md` with the labels in fixture-notes.
- With the fixture loaded, spot-check two commitments in the detail page’s Why? timeline.
- Answer Claude’s question about committing the fixture.

S03 · Deadline parser and message signals

Essential · Replaces C04 + C05

Goal. Cover the deadline forms the plan requires but the parser misses, record owner replies like “still working on it” as evidence without changing status, reopen retracted completions, and add an extractor seam for optional AI.

```
PROMPT S03 - Deadline parser and message signals

You are working on FollowThru (Fynd Hackathon 2026, challenge C14,
"Chase what's late. Never what's done.") in D:\followthru.
Single-laptop workflow: one developer, one branch, no Track A / Track B split.

BEFORE EDITING
1. Read CLAUDE.md, README.md, docs/integration-status.md and
   docs/fixture-notes.md.
2. Run: git status --short ; git branch --show-current ; git log --oneline -8
3. Work only on branch full-integration (git checkout full-integration ;
   git pull origin full-integration). If git status shows changes you did
   not make in this session, STOP and show them. Never stash, reset or
   discard. Leave docs/TRACK_A_BACKEND_PROMPT.md untracked and untouched.
4. Run npm test once. If it fails before you change anything, report first.

PROJECT FACTS AND RULES
All facts and rules in CLAUDE.md apply. In short: src/engine/types.ts is the
single contract (additive changes only, tests/contract.test.ts updated on
purpose, changes logged in docs/integration-status.md); src/engine is pure
(no Date.now, clock = EngineConfig.now); UI uses src/store hooks only; every
write is audit-logged via src/store/repo.ts; silence never changes status;
ambiguous evidence -> needsReview; one versioned draft per commitment; zero
active drafts on done, cancelled or deferred work; shadow mode only; never
invent labels, fields or accuracy; synthetic data is always labelled.

GIT AND REPORTING RULES
- test + typecheck + build pass before every commit; review
  git diff --staged; stage by explicit path, never git add -A or git add .
- Commits: lowercase, 3-8 words, no symbols, no prefixes, one line. No
  Co-Authored-By and no AI or Claude mention anywhere. Never change git
  config or use --author.
- No node_modules, dist, .env, keys, keystores, APKs or personal data.
- No force push, reset --hard, clean, checkout -- ., rebase of pushed
  branches or branch deletion. Never merge into main without the exact
  phrase: merge into main. Push after each commit:
  git push origin full-integration
- Never claim results you did not produce; never weaken tests.

CURRENT CONTEXT (verified at cad765d; re-check against the code)
src/engine/deadline.ts parseDeadline(text, { saidAt, tz }) supports weekdays
with this/next/coming, today, tonight, tomorrow, day after tomorrow,
EOD/COB/end of day (18:00), end of week (Friday 18:00), ISO dates, month-name
dates, times (6pm, 18:00, noon, morning, afternoon, evening) and bare hours
with -ish. Date-only -> 23:59 local (DATE_ONLY_TIME). Hedges in HEDGE_RE make
certainty tentative. src/engine/time.ts wallTimeToUtcMs handles DST (gap
moves forward, overlap takes the later instant). tests/deadline.test.ts and
tests/time.test.ts: 33 passing cases. Rules table: docs/track-a-status.md.
src/engine/signals.ts detectSignals returns request, commit, accept, done,
cancel, defer, reschedule, delegate, decline (DONE_NEGATED stops "not sent
```

```
yet"). history.ts applyUpdate applies them with authority checks. An owner
"still working on it" only updates lastOwnerMsgAt (responseState responded),
no event. No reopen path. CommitmentEventType has no progress, not_done or
blocker. Blockers come only from report_blocker decisions. Detail page
labels events via EVENT_LABEL in src/pages/CommitmentDetail.tsx.

PART 1 - DEADLINE PARSER (every existing test passes unchanged)
1. Explicit timezone in the phrase overrides the speaker's: "5pm IST",
   "9am PT", "14:00 UTC", "10am GMT", "6pm BST", "9am ET", "4pm CET",
   "11am SGT". Add an abbreviation map in src/engine/time.ts: IST ->
   Asia/Kolkata, PT/PST/PDT -> America/Los_Angeles, ET/EST/EDT ->
   America/New_York, UTC and GMT -> UTC, BST -> Europe/London, CET/CEST ->
   Europe/Paris, SGT -> Asia/Singapore. Document IST is read as India.
   interpretedTz returns the zone actually used. A weekday phrase's
   calendar day is resolved from the speaker's "today", then the time is
   applied in the explicit zone (document in code comment and table).
2. Relative durations: "in 2 days", "in 3 hours", "in a week", "within 48
   hours". Days keep the date-only end-of-day rule; hours are exact.
3. "next week" -> Monday of next week, date-only rule (23:59 local).
   "by end of next week" -> Friday of next week, 18:00.
4. Numeric dates "12/10", "12/10/2026", "12-10-2026" read as day/month;
   if invalid as day/month but valid as month/day use month/day. Year via
   the existing closestYear helper.
5. More tentative hedges: "i think", "around", "roughly", "approx",
   "approximately", and a "?" directly after the date or time ("Friday?").
   "at the latest" and "by Friday sharp" stay firm (negative tests).
6. Update the deadline rules table in docs/track-a-status.md with every new
   rule and assumption.
7. At least 25 new cases in tests/deadline.test.ts, including: "by Monday
   10am PT" from an IST speaker uses Pacific; a late-Thursday-UTC message
   that is already Friday in IST; "tomorrow EOD" across a month end; "in 2
   days" across a DST change in America/Los_Angeles; "12/10" vs "13/10" vs
   "10/13"; each new hedge plus the firm negatives; all old cases unchanged.
8. tests/engine.test.ts must pass without edits; if a demo expectation
   changes, stop and explain. Export new helpers from src/engine/index.ts
   only if another module needs them (then update tests/contract.test.ts).
   Commits: read explicit timezones in deadline phrases / parse relative and
   numeric dates / widen tentative deadline hedges

PART 2 - MESSAGE SIGNALS (no status change from silence or progress notes)
9. Types (additive): add 'progress', 'not_done', 'blocker' to
   CommitmentEventType.
10. src/engine/signals.ts new kinds:
    - not_done: "not yet", "still working", "running late", "need more
      time", "not done", "not finished";
    - blocker: "blocked by", "waiting on", "can't until", "stuck on",
      "waiting for";
    - progress: "making progress", "half way", "halfway", "almost there",
      "first draft ready", "nearly done";
    - reopen: "not done after all", "still broken", "reopening", "that was
      the wrong file", "please redo", "actually it's not done".
    Keep DONE_NEGATED. "almost done" is progress, never done.
11. history.ts applyUpdate:
    - owner (or accepted delegate) not_done, progress or blocker messages
      add the matching event with evidence; status unchanged;
    - blocker messages also append Blocker { at, by, text: short quote,
      resolved: false } to commitment.blockers;
    - reopen after done by the requester, a coordinator or the owner ->
      status open, event 'reopened' with evidence; by anyone else ->
      needsReview with reason 'conflicting_evidence';
```

```
    - drafts.ts already reactivates a suppressed draft as a new version
      when work is open and overdue: verify it, do not duplicate it.
12. Extractor seam: src/engine/extractor.ts with
    export interface Extractor { name: 'rules' | 'ai';
    detect(text: string): Signal[] } and export const rulesExtractor.
    resolveCommitments and runEngine accept an optional extractor that
    defaults to rulesExtractor. No behaviour change for existing callers.
13. UI labels in EVENT_LABEL (CommitmentDetail.tsx): Progress update, Not
    done yet, Blocker reported.
14. Tests in tests/engine.test.ts (withMessages helper, synthetic messages):
    "still working on it, need two more days" after the deadline -> status
    open, responseState responded, not_done event, still drafted; blocker
    message adds a blocker and an event; "almost done" is not completion;
    reopen by the requester reactivates the SAME draft id with a higher
    version and cause 'reactivated'; reopen by an unrelated person goes to
    review; an injected fake extractor is used when passed.
15. Update tests/contract.test.ts only if index.ts exports change; log the
    type additions in docs/integration-status.md.
    Commits: detect progress and blocker messages / reopen work when
    completion is retracted / add extractor interface for rules

TESTING
After each part: npm test, npm run typecheck, npm run build,
npm run evidence -- --demo (synthetic scorecard must still show 3 drafts and
0 active reminders on closed work, checks 2 and 3 PASS, unless you can
explain a change). npm run e2e at the end.

ACCEPTANCE CRITERIA
- 25+ new deadline tests pass; all previous tests unchanged; interpretedTz
  reflects an explicit zone; rules table updated.
- New signals detected and tested including the "almost done" negative;
  progress or not-done replies never change status; reopen follows
  authority rules; one draft per commitment holds; runEngine works with and
  without an explicit extractor; contract changes logged.
- Demo evidence unchanged (3 drafts, checks 2 and 3 PASS); all suites pass.

FINAL REPORT
Files changed; real results of every command (test counts, typecheck, build
sizes, evidence summary, e2e); commits with hashes;
git log -6 --format="%an <%ae>%n%b" output; anything not done and why.
If data/fixture.json exists, remind me to re-run S02 Part 2. Then stop.
```

You check

- `npm test`: the new deadline cases and signal tests appear by name.
- `npm run evidence -- --demo`: still 3 drafts, PASS for reminder and duplicate checks.
- `npm run dev`, load the demo, open a commitment's detail page: the timeline still renders.

S04 · Reminder wording, escalation and product change record

Essential · Replaces C06 + C07

Goal. Drafts say different things to someone who never replied and someone who said “not yet”, get short subjects, and can escalate to the requester or coordinator while keeping one draft per commitment. Then make the feature flags produce a real before/after comparison and save the starting state, negotiated scope and final evidence the brief requires.

```
PROMPT S04 - Reminder wording, escalation and product change record

You are working on FollowThru (Fynd Hackathon 2026, challenge C14,
"Chase what's late. Never what's done.") in D:\followthru.
Single-laptop workflow: one developer, one branch, no Track A / Track B split.

BEFORE EDITING
1. Read CLAUDE.md, README.md, docs/integration-status.md and
   docs/fixture-notes.md.
2. Run: git status --short ; git branch --show-current ; git log --oneline -8
3. Work only on branch full-integration (git checkout full-integration ;
   git pull origin full-integration). If git status shows changes you did
   not make in this session, STOP and show them. Never stash, reset or
   discard. Leave docs/TRACK_A_BACKEND_PROMPT.md untracked and untouched.
4. Run npm test once. If it fails before you change anything, report first.

PROJECT FACTS AND RULES
All facts and rules in CLAUDE.md apply. In short: src/engine/types.ts is the
single contract (additive changes only, tests/contract.test.ts updated on
purpose, changes logged in docs/integration-status.md); src/engine is pure
(no Date.now, clock = EngineConfig.now); UI uses src/store hooks only; every
write is audit-logged via src/store/repo.ts; silence never changes status;
ambiguous evidence -> needsReview; one versioned draft per commitment; zero
active drafts on done, cancelled or deferred work; shadow mode only; never
invent labels, fields or accuracy; synthetic data is always labelled.

GIT AND REPORTING RULES
- test + typecheck + build pass before every commit; review
  git diff --staged; stage by explicit path, never git add -A or git add .
- Commits: lowercase, 3-8 words, no symbols, no prefixes, one line. No
  Co-Authored-By and no AI or Claude mention anywhere. Never change git
  config or use --author.
- No node_modules, dist, .env, keys, keystores, APKs or personal data.
- No force push, reset --hard, clean, checkout -- ., rebase of pushed
  branches or branch deletion. Never merge into main without the exact
  phrase: merge into main. Push after each commit:
  git push origin full-integration
- Never claim results you did not produce; never weaken tests.

CURRENT CONTEXT (verified at cad765d; re-check against the code)
drafts.ts composeBody(c, kind, tone, people, now) ignores responseState; no
subject. Escalate tone mentions the requester but recipientId is always the
owner. approval.ts checkBeforeApproval treats draft.recipientId !==
commitment.ownerId as stale. FlowStep is { afterHours, tone }; DEFAULT_FLOW
in flows.ts: 0h gentle, 24h firm, 72h escalate. Flows page edits steps;
Queue page shows body and recipient.
scripts/evidence.ts supports --input, --demo, --now and writes
evidence/<demo|run>-<stamp>/ with scorecard.md, results.json,
commitments.csv, drafts.csv, validation.json, manifest.json (SHA-256 per
```

```
file). No --label, flag switch, evidence/latest, test report, git hash or
suppressions CSV. .gitignore ignores evidence/demo-*/. EngineFlags
{ staleSuppression, recipientReview, ai } exist, but buildDrafts always
suppresses closed-work drafts and checkBeforeApproval always re-checks
freshness; only time-based staleness and recipient confirmation honour the
flags. No docs/product-change.md.

PART 1 - WORDING, SUBJECTS AND ESCALATION RECIPIENT
1. Types (additive): Draft.subject?: string; DraftVersion.subject?: string;
   FlowStep.recipient?: 'owner' | 'requester' | 'coordinator' (missing =
   owner).
2. src/engine/drafts.ts:
   - composeSubject(...) under 60 characters, e.g. "Quick check: <task>"
     or "Date needed: <task>".
   - composeBody variants, each under 70 words, polite, specific, using the
     recipient's first name and the deadline in the recipient's timezone:
     a. reminder + responseState pending: "haven't heard back";
     b. reminder + latest event not_done or progress: ask for a realistic
        new date;
     c. clarify: confirm a firm date for the tentative phrase;
     d. escalation to requester or coordinator: factual summary naming the
        owner, the deadline, how long overdue and the last reply time.
   - export expectedRecipient(c, step, people): owner by default;
     'requester' -> c.requesterId, falling back to the coordinator;
     'coordinator' -> first person with role coordinator sorted by id.
   - buildDrafts uses expectedRecipient. A recipient change between steps
     already bumps the version with cause 'recipient_changed' and resets
     recipientConfirmed - verify with a test.
3. approval.ts: replace the ownerId comparison with expectedRecipient so
   escalation drafts are not wrongly stale, while a changed owner on an
   owner-addressed draft is still stale.
4. Keep DEFAULT_FLOW unchanged (all steps to owner). Add the recipient
   choice to the step editor in src/pages/Flows.tsx and to validateFlow
   (escalating to a requester when there is none is allowed but warned in
   the preview).
5. Optional if time allows: quiet hours. Flow gains optional quietHours
   { start: 'HH:mm', end: 'HH:mm' } in the recipient's timezone; a step due
   inside quiet hours waits until they end. Pure, tested, editable in Flows.
   Skip it if any core test becomes hard to keep green, and say so.
6. Queue card shows the subject above the body.
7. Tests: wording differs for pending vs not_done; subject length; a custom
   flow escalating to the coordinator at step 2 gives the same draft id,
   version + 1, recipient = coordinator, recipientConfirmed false, and
   approval passes after confirmation; owner-change staleness still
   detected; demo expectations unchanged. Update contract test and
   integration-status as needed. Run npm run e2e (queue approval journey).
   Commits: word reminders by reply state / add draft subjects / let flow
   steps escalate to coordinator

PART 2 - PRODUCT CHANGE RECORD (defaults stay ON everywhere in the app)
8. Flag semantics in the engine:
   - staleSuppression = false (baseline): buildDrafts keeps a previously
     created draft unchanged when its situation later changes (no
     auto-suppression, no time-based stale marking); drafts are still never
     CREATED for closed work. checkBeforeApproval skips the freshness
     re-check (terminal status, changed deadline or owner, age) and only
     checks draft state.
   - recipientReview = false: approval does not need recipientConfirmed.
   - Tests: with staleSuppression false a draft created before a completion
     message stays pending_review after it (the baseline problem); with true
```

```
     it is suppressed with reason completed and evidence. Default-config
     result for the demo must be byte-identical to before this part.
9. scripts/evidence.ts:
   - --label <name>: fixture runs -> evidence/runs/<YYYY-MM-DD_HH-mm-ss>_<label>/;
     synthetic runs -> evidence/demo-runs/<same>/ (git-ignored unless
     copied into starting_state or final).
   - --flags on|off (default on).
   - --simulate-later <ISO cutoff>: run 1 with messages at or before the
     cutoff and now = cutoff; run 2 with all messages and run 1's drafts as
     previousDrafts; write stale-report.md listing every draft that became
     outdated (and, with flags off, which would still be approved and sent
     wrongly).
   - test-report.txt: output of npx vitest run --reporter=verbose; record a
     failure honestly and exit non-zero.
   - suppressions.csv via new suppressionsToCsv(drafts) in src/schema/io.ts
     (draft id, commitment id, reason, at, evidence message id),
     formula-escaped like other exports, unit test in tests/io.test.ts,
     exported from src/schema/index.ts.
   - manifest.json adds label, flags, engine mode, dataset name, synthetic
     flag, git commit hash and dirty-tree flag (git rev-parse --short HEAD,
     git status --porcelain).
   - copy the run to evidence/latest/ (replace its contents).
10. Runs (data/fixture.json if present, otherwise --demo and label
    everything SYNTHETIC in titles and READMEs):
    - starting state: --label starting-state --flags off --simulate-later
      with a cutoff before the completion messages (demo: 2026-10-16T08:00:00Z,
      before the cross-thread completion m_10); copy to
      evidence/starting_state/ with README.md: how many drafts, for which
      commitments, which became outdated, recipients not reviewed.
    - final: same command with --flags on, label final; copy to
      evidence/final/ with README.md comparing before and after.
11. docs/product-change.md: starting state (link), the problem (stale
    drafts, wrong recipients), scope IN (pre-send freshness check, automatic
    suppression or update with visible reason, staleness after
    staleAfterHours, recipient review with warnings and confirmation, both
    toggles for comparison), scope OUT (real sending, Slack or email
    integration, ML ranking, multi-recipient drafts), numbered acceptance
    criteria each mapped to a test name, and a Final evidence section.
12. Commit evidence/starting_state/, evidence/final/ and evidence/latest/
    even when synthetic (their READMEs say so). Update .gitignore as
    needed. Run npm run e2e.
    Commits: make stale suppression follow its flag / add labelled evidence
    runs with test report / record product change scope and evidence

ACCEPTANCE CRITERIA
- Different tested wording for pending, not done, clarify and escalation;
  escalation keeps one draft per commitment and approves after confirmation;
  owner-change staleness still blocks; DEFAULT_FLOW and demo results
  unchanged.
- Flags off reproduces the stale-draft problem, flags on fixes it, both
  tested; default results unchanged.
- evidence/starting_state, evidence/final and evidence/latest exist with
  manifest hashes, a test report and a clearly stated data source.
- docs/product-change.md has starting state, scope IN and OUT, acceptance
  criteria mapped to tests, and final evidence links. All suites pass.

FINAL REPORT
Files changed; real results of every command (test counts, typecheck, build
sizes, both evidence runs, e2e); commits with hashes;
```

```
git log -6 --format="%an <%ae>%n%b" output; whether quiet hours was done;
anything not done and why. Then stop and wait.
```

You check

- In the app, open Flows, create a flow for everyone with step 2 escalating to the coordinator, then check Queue for the recipient and subject. Delete the test flow afterwards.
- Open `docs/product-change.md` and both evidence READMEs: they must say SYNTHETIC if the fixture was not used.
- Compare `evidence/final/stale-report.md` with `evidence/starting_state/stale-report.md`.

S05 · Draft review actions and role-based navigation

Essential · Replaces C08 + C09

Goal. Coordinators can reject or edit drafts (audited, preserved across engine runs) and see tentative-deadline drafts in their own lane. Each role sees only the screens it can use, members can ask for more time (never auto-applied), and coordinators get a people grid and an extension inbox.

```
PROMPT S05 - Draft review actions and role-based navigation

You are working on FollowThru (Fynd Hackathon 2026, challenge C14,
"Chase what's late. Never what's done.") in D:\followthru.
Single-laptop workflow: one developer, one branch, no Track A / Track B split.

BEFORE EDITING
1. Read CLAUDE.md, README.md, docs/integration-status.md and
   docs/fixture-notes.md.
2. Run: git status --short ; git branch --show-current ; git log --oneline -8
3. Work only on branch full-integration (git checkout full-integration ;
   git pull origin full-integration). If git status shows changes you did
   not make in this session, STOP and show them. Never stash, reset or
   discard. Leave docs/TRACK_A_BACKEND_PROMPT.md untracked and untouched.
4. Run npm test once. If it fails before you change anything, report first.

PROJECT FACTS AND RULES
All facts and rules in CLAUDE.md apply. In short: src/engine/types.ts is the
single contract (additive changes only, tests/contract.test.ts updated on
purpose, changes logged in docs/integration-status.md); src/engine is pure
(no Date.now, clock = EngineConfig.now); UI uses src/store hooks only; every
write is audit-logged via src/store/repo.ts; silence never changes status;
ambiguous evidence -> needsReview; one versioned draft per commitment; zero
active drafts on done, cancelled or deferred work; shadow mode only; never
invent labels, fields or accuracy; synthetic data is always labelled.

GIT AND REPORTING RULES
- test + typecheck + build pass before every commit; review
  git diff --staged; stage by explicit path, never git add -A or git add .
- Commits: lowercase, 3-8 words, no symbols, no prefixes, one line. No
  Co-Authored-By and no AI or Claude mention anywhere. Never change git
  config or use --author.
- No node_modules, dist, .env, keys, keystores, APKs or personal data.
- No force push, reset --hard, clean, checkout -- ., rebase of pushed
  branches or branch deletion. Never merge into main without the exact
  phrase: merge into main. Push after each commit:
  git push origin full-integration
- Never claim results you did not produce; never weaken tests.

CURRENT CONTEXT (verified at cad765d; re-check against the code)
DraftState is pending_review | approved | suppressed | stale.
src/store/actions.ts has approve(draftId) (runs checkBeforeApproval,
suppresses when stale) and confirmRecipient. Queue tabs: Needs review,
Approved, Stale, Suppressed; clarify drafts sit in Needs review. Approved
state survives runs because buildDrafts keeps an unchanged previous draft.
draftStateView in src/components/domain/view.ts maps states to chips;
evaluate.ts treats pending_review and approved as active. syncDrafts in
repo.ts writes whatever the engine returns, so human states must be
preserved by buildDrafts itself.
```

AppShell guards for data and profile. Sidebar.tsx, BottomTabBar.tsx and MobileNavSheet.tsx show all routes to everyone. Queue filters drafts to a member's own (read only). Team.tsx has coordinator (assign, submissions, blockers) and member views. Member actions (progress, blocker, submit) are in CommitmentDetail.tsx. Permissions: canDecide and canDo in permissions.ts. Decisions applied in applyDecision in history.ts. All decisions go through createActions.decide in actions.ts.

PART 1 - REJECT, EDIT AND CLARIFICATION LANE

1. Types (additive): DraftState adds 'rejected'. Draft adds optional rejectedBy, rejectedAt, rejectReason, editedBody, editedBy, editedAt. Fix every exhaustive mapping TypeScript flags (view.ts, Queue, evaluate ACTIVE list keeps rejected inactive).
2. Pure functions in approval.ts: rejectDraft(draft, by, at, reason) and editDraft(draft, by, at, body). Body 1 to 600 characters, trimmed.
3. buildDrafts: a rejected or edited draft stays as it is while nothing changes; when a cause creates a new version the draft returns to pending_review, the edit is dropped and history says "edit replaced by new version (<cause>)". Approval and exports use editedBody when present. Respect the staleSuppression and recipientReview flags added in S04.
4. actions.ts: reject(draftId, reason) and edit(draftId, body), coordinator only (canDo 'approve_draft'), reason required for reject, audited through repo.putDraft with actions draft.reject and draft.edit.
5. Queue.tsx:
   - Reject opens a Dialog with a required reason.
   - Edit switches the body to a Textarea with Save and Cancel; edited drafts show an "Edited" note.
   - Lanes: Needs review (reminders pending), Needs clarification (clarify pending), Approved (shadow), Stale, Suppressed, Rejected.
   - Header from the scorecard: "<n> active reminders for completed, cancelled or deferred work" (should be 0).
   - Optional: shortcuts J/K next/previous, A approve, R reject, E edit, only when no input has focus.
6. Tests: engine (rejected survives re-run; new version returns to review; edit preserved then replaced); store (member cannot reject or edit; reason required); tests/ui.test.tsx (reject moves the card to Rejected); extend e2e/app.spec.ts with an edit then reject step. Update contract test and integration-status.
   Commits: add reject and edit for drafts / show clarification lane in queue

PART 2 - ROLE NAVIGATION, EXTENSIONS AND PEOPLE GRID

7. One shared nav list in src/components/shell/nav.ts used by Sidebar, BottomTabBar and MobileNavSheet:
   - coordinator: Dashboard, Commitments, Queue (badge = pending review count), Flows, Team, Evidence;
   - member: My work (dashboard), My tasks (/team), Commitments, Evidence.
8. src/components/shell/RequireRole.tsx: members opening /queue or /flows are redirected to /team with a short notice. Use it in src/App.tsx.
9. "Need more time" (never auto-applied):
   - types (additive): DecisionType adds 'request_extension' and 'decline_extension'; CommitmentEventType adds 'extension_requested'; Commitment adds optional extensionRequest { dueAt, by, at, note, state: 'pending' | 'approved' | 'declined' }.
   - permissions: the owner (member) can request_extension; only a coordinator can decline_extension; approving is a change_deadline decision that also marks the pending request approved.
   - applyDecision handles both; a pending request does not change the deadline or status and does not stop reminders.
   - CommitmentDetail: member form (new date in their time + reason);

```
     coordinator sees Approve new date / Decline.
10. Team page (coordinator): people grid (name, team, timezone, local time
    at the evaluation clock, open, overdue, done, open blockers) and an
    extension requests inbox.
11. Optional: small approval celebration in the member view with Framer
    Motion, off when prefers-reduced-motion is set.
12. Tests: permissions unit tests; engine decisions (request keeps the
    deadline, approve changes it and bumps the draft version, member cannot
    decline); UI (member nav has no Queue link, /queue redirects); e2e:
    member requests an extension, coordinator approves, deadline and draft
    version change.
    Commits: show navigation by role / let members request more time / add
    people grid and extension inbox

TESTING
After each part: npm test, npm run typecheck, npm run build, npm run e2e.

ACCEPTANCE CRITERIA
- Reject needs a reason, edit is validated, both audited and
  coordinator-only, both survive engine re-runs until a new version.
- Six lanes visible; header count shows 0 for the demo.
- Members never see Queue or Flows and are redirected if they type the URL.
- Extension requests never change a deadline until a coordinator approves.
- People grid shows local times at the evaluation clock. All suites pass.

FINAL REPORT
Files changed; real results of every command (test counts, typecheck, build
sizes, e2e); commits with hashes; git log -6 --format="%an <%ae>%n%b"
output; which optional items were done; anything not done and why. Then stop.
```

You check

- As Priya, edit a draft, reload: the edit stays. Reject another with a reason; check the Rejected lane and Evidence → Audit log.
- Switch profile to Lena: no Queue or Flows in the sidebar; typing `/queue` sends you to My tasks.
- As Lena request more time on the press kit; switch to Priya, approve it, and check the Why? timeline.

S06 · Backup, evidence pack, audit filters and REST API

Essential · Replaces C10 + C11

Goal. Make local data portable and evidence easy to hand over (backup and restore, one evidence pack download, audit filters, audited evidence deletion), and finish the API (CSV input, evidence downloads, CORS, written docs) reusing the same engine.

```
PROMPT S06 - Backup, evidence pack, audit filters and REST API

You are working on FollowThru (Fynd Hackathon 2026, challenge C14,
"Chase what's late. Never what's done.") in D:\followthru.
Single-laptop workflow: one developer, one branch, no Track A / Track B split.

BEFORE EDITING
1. Read CLAUDE.md, README.md, docs/integration-status.md and
   docs/fixture-notes.md.
2. Run: git status --short ; git branch --show-current ; git log --oneline -8
3. Work only on branch full-integration (git checkout full-integration ;
   git pull origin full-integration). If git status shows changes you did
   not make in this session, STOP and show them. Never stash, reset or
   discard. Leave docs/TRACK_A_BACKEND_PROMPT.md untracked and untouched.
4. Run npm test once. If it fails before you change anything, report first.

PROJECT FACTS AND RULES
All facts and rules in CLAUDE.md apply. In short: src/engine/types.ts is the
single contract (additive changes only, tests/contract.test.ts updated on
purpose, changes logged in docs/integration-status.md); src/engine is pure
(no Date.now, clock = EngineConfig.now); UI uses src/store hooks only; every
write is audit-logged via src/store/repo.ts; silence never changes status;
ambiguous evidence -> needsReview; one versioned draft per commitment; zero
active drafts on done, cancelled or deferred work; shadow mode only; never
invent labels, fields or accuracy; synthetic data is always labelled.

GIT AND REPORTING RULES
- test + typecheck + build pass before every commit; review
  git diff --staged; stage by explicit path, never git add -A or git add .
- Commits: lowercase, 3-8 words, no symbols, no prefixes, one line. No
  Co-Authored-By and no AI or Claude mention anywhere. Never change git
  config or use --author.
- No node_modules, dist, .env, keys, keystores, APKs or personal data.
- No force push, reset --hard, clean, checkout -- ., rebase of pushed
  branches or branch deletion. Never merge into main without the exact
  phrase: merge into main. Push after each commit:
  git push origin full-integration
- Never claim results you did not produce; never weaken tests.

CURRENT CONTEXT (verified at cad765d; re-check against the code)
repo.ts has loadDataset, resetAllData (keeps the audit log), addDecision,
addTasks, syncDrafts, putDraft, saveFlow, deleteFlow, addEvidenceFile
(SHA-256), saveRun. db.ts tables: settings, datasets, decisions, tasks,
drafts, flows, audit, evidenceFiles, runs. Evidence page tabs: Scorecard,
Exports, Uploads, Audit log (latest 200, no filters), Data & settings;
Export data downloads only the dataset; evidence files cannot be deleted.
EngineProvider watches tables with useLiveQuery.
api/_lib/handlers.ts: handleHealth (GET), handleProcess (POST
{ dataset, config? }, JSON only, 2 MB limit, error shape
{ error: { code, message, issues? } }, logs only status and timing).
```

api/health.ts and api/process.ts wrap them. vite.config.ts dev plugin serves /api/health and /api/process. vercel.json rewrites non-API paths to index.html. tests/api.test.ts: 13 tests. No /api/evidence, CORS, CSV input or docs/api.md.

PART 1 - BACKUP, RESTORE, EVIDENCE PACK, AUDIT

1. repo.exportBackup(): JSON { format: 'followthru-backup', version: 1, exportedAt, dataset, decisions, tasks, drafts, flows, settings, runs, audit, evidenceFiles: metadata only (no blobs) }. Exclude any secret setting (for example an AI key added later).
2. Zod backup schema in src/schema/backup.ts reusing existing schemas; the dataset part must also pass validateDataset. repo.importBackup(json) validates, then replaces all tables except evidence blobs in one transaction and writes audit 'data.restore'. Coordinator only (or first run with no data).
3. Evidence pack: one JSON download { generatedAt, dataset name, source, synthetic flag, evaluation clock, engine mode, flags, scorecard, commitments, drafts, audit, evidence file metadata with sha256 }; show the SHA-256 of the downloaded content next to the button (repo.sha256Hex).
4. Audit log: filters for actor, action prefix, entity and date range (dates in UTC and local), "Load more" paging, before/after view that highlights changed keys. Extend useAuditLog in src/store/hooks.ts with a filter argument; do not filter huge arrays in the component.
5. Evidence file deletion: coordinator only, reason required, audit entry keeps name, size and sha256.
6. UI: Data & settings gets Export backup and Restore backup (file input -> validation summary -> confirm Dialog); Exports gets Download evidence pack.
7. Tests (tests/store.test.ts): export -> reset -> import gives the same decisions, tasks, drafts (versions and states) and flows; invalid backup rejected with issues; member cannot restore or delete; delete needs a reason. UI test for an audit filter.
   Commits: add full backup and restore / add evidence pack download / add audit filters and file delete

PART 2 - REST API COMPLETION

8. handleProcess also accepts { csv, people, threads? }: build the dataset with messagesFromCsv (src/schema/io.ts) and validateDataset. If src/schema/adapter.ts exists (S02), accept { fixture } and adapt it.
9. handleEvidence (GET): format=json|csv and kind=commitments|drafts|suppressions|scorecard. Source: data/fixture.json when it exists (read with fs at runtime; add "functions": { "api/evidence.ts": { "includeFiles": "data/fixture.json" } } to vercel.json only if the fixture is committed), otherwise the synthetic demo with header X-FollowThru-Dataset: synthetic-demo and dataset: "synthetic demo (not challenge data)" in JSON.
   Content-Disposition attachment with a timestamped file name. 400 for bad parameters, 405 for other methods. Use suppressionsToCsv from S04 (add it if missing).
10. CORS helper used by all routes: Access-Control-Allow-Origin *, methods GET, POST, OPTIONS; OPTIONS returns 204.
11. api/evidence.ts wrapper; add the route and OPTIONS to the dev plugin in vite.config.ts. Handlers import src/engine and src/schema directly; no duplicate logic.
12. docs/api.md: every endpoint, request and response shapes from the real types (EngineResult, Scorecard, ValidationIssue), error table (400, 405, 413, 415, 422), curl examples for http://localhost:5173 and for LIVE_URL (a clearly marked placeholder), and notes: shadow mode, no message text in logs, synthetic fallback header. Link it from README.md.

```
13. Tests: tests/api.test.ts for CSV input, evidence kinds and formats, bad
    parameters, OPTIONS; e2e/app.spec.ts checks /api/evidence in the dev
    server. Actually run the curl examples against npm run dev.
    Commits: accept csv input in process api / add evidence api with cors /
    add api docs

TESTING
After each part: npm test, npm run typecheck, npm run build, npm run e2e.

ACCEPTANCE CRITERIA
- Backup and restore round-trip passes in tests and in the browser.
- Evidence pack downloads with a visible SHA-256.
- Audit filters work; evidence deletion needs a reason and is audited.
- /api/health, /api/process (JSON and CSV) and /api/evidence work in dev and
  tests; synthetic fallback labelled in headers and body.
- docs/api.md examples were actually run and match the responses.

FINAL REPORT
Files changed; real results of every command (test counts, typecheck, build
sizes, e2e, curl outputs); commits with hashes;
git log -6 --format="%an <%ae>%n%b" output; anything not done and why.
Then stop and wait.
```

You check

- Approve a draft, export a backup, reset all data, restore the backup: the approval is back.
- Download the evidence pack and open it in a text editor.
- With `npm run dev` running, open `http://localhost:5173/api/evidence?format=csv&kind=drafts`: a file downloads. Run one curl example from `docs/api.md`.

S07 · PWA completion

Essential (required for the APK) · Replaces C12

Goal. Make FollowThru properly installable and verifiably offline, with the PNG icons and manifest fields PWABuilder needs.

```
PROMPT S07 - PWA completion: icons, manifest, install, offline and update

You are working on FollowThru (Fynd Hackathon 2026, challenge C14,
"Chase what's late. Never what's done.") in D:\followthru.
Single-laptop workflow: one developer, one branch, no Track A / Track B split.

BEFORE EDITING
1. Read CLAUDE.md, README.md, docs/integration-status.md and
   docs/fixture-notes.md.
2. Run: git status --short ; git branch --show-current ; git log --oneline -8
3. Work only on branch full-integration (git checkout full-integration ;
   git pull origin full-integration). If git status shows changes you did
   not make in this session, STOP and show them. Never stash, reset or
   discard. Leave docs/TRACK_A_BACKEND_PROMPT.md untracked and untouched.
4. Run npm test once. If it fails before you change anything, report first.

PROJECT FACTS AND RULES
All facts and rules in CLAUDE.md apply. In short: src/engine/types.ts is the
single contract (additive changes only); src/engine is pure; UI uses
src/store hooks only; every write is audit-logged; shadow mode only; never
invent labels, fields, accuracy or scores; synthetic data is labelled.

GIT AND REPORTING RULES
- test + typecheck + build pass before every commit; review
  git diff --staged; stage by explicit path, never git add -A or git add .
- Commits: lowercase, 3-8 words, no symbols, no prefixes, one line. No
  Co-Authored-By and no AI or Claude mention anywhere. Never change git
  config or use --author.
- No node_modules, dist, .env, keys, keystores, APKs or personal data.
- No force push, reset --hard, clean, checkout -- ., rebase of pushed
  branches or branch deletion. Never merge into main without the exact
  phrase: merge into main. Push after each commit:
  git push origin full-integration
- Never claim results you did not produce; never weaken tests.

CURRENT CONTEXT (verified at cad765d; re-check against the code)
vite.config.ts configures VitePWA (registerType autoUpdate, name FollowThru,
start_url /dashboard, one SVG icon /favicon.svg, workbox globPatterns,
navigateFallbackDenylist for /api). public/ has only favicon.svg. index.html
has theme-color and viewport-fit=cover. No PNG icons, apple-touch-icon,
install button, offline banner or update prompt. Build precaches 18 entries.
Offline never tested. Data lives in IndexedDB, so the app works offline once
the shell is cached; the demo dataset is bundled; the fixture only if
committed.

TASK
1. Original logo public/logo.svg: a check mark merging into a forward arrow,
   violet #7C5CFF to mint #3EE6B0 on #07060D. Generate pwa-192x192.png,
   pwa-512x512.png, maskable-512x512.png (safe-zone padding) and
   apple-touch-icon-180x180.png with @vite-pwa/assets-generator (dev
   dependency) and pwa-assets.config.ts; add script "icons". Open the
```

```
   generated PNGs to confirm they render.
2. Manifest: name "FollowThru - Chaser Draft Queue", short_name
   "FollowThru", description, start_url "/dashboard", scope "/", display
   "standalone", background_color and theme_color "#07060D", orientation
   "any", categories ["productivity", "business"], icons with correct
   purposes, shortcuts (Queue /queue, Dashboard /dashboard, My tasks /team),
   screenshots: one wide 1280x720 and one narrow 390x844 PNG in
   public/screenshots/ captured with Playwright (installed Chrome) from the
   demo data, with form_factor wide and narrow.
3. index.html: apple-touch-icon link, apple-mobile-web-app-capable,
   apple-mobile-web-app-status-bar-style black-translucent.
4. src/components/shell/InstallButton.tsx: beforeinstallprompt on Android
   and desktop; on iOS a Dialog with Share -> Add to Home Screen. Show it in
   the TopBar and on the Landing page.
5. Offline banner in AppShell (navigator.onLine plus online/offline events).
6. Update prompt with useRegisterSW from virtual:pwa-register/react (add the
   vite-plugin-pwa/react types to tsconfig.app.json): "Update available -
   Reload".
7. Verification: playwright.pwa.config.ts serving the production build with
   npm run build && npm run preview -- --port 4173 --strictPort, script
   "e2e:pwa", and e2e/pwa.spec.ts: manifest reachable with the fields above,
   icons return 200, a service worker registers, and after one visit
   context.setOffline(true) then reload of /dashboard still renders. Run
   Lighthouse only if available locally; never report a score you did not
   run.

TESTING
npm test, npm run typecheck, npm run build, npm run e2e, npm run e2e:pwa.

COMMITS (push after each)
add app logo and icons / complete pwa manifest / add install button and
offline banner / add update prompt / add pwa browser test

ACCEPTANCE CRITERIA
- PNG icons (192, 512, maskable 512, apple 180) exist and render.
- e2e:pwa passes: manifest fields, service worker, offline reload.
- Install button works in Chrome desktop; no console errors.

FINAL REPORT
Files changed; real results of every command (test counts, typecheck, build
sizes and precache count, e2e, e2e:pwa); commits with hashes;
git log -5 --format="%an <%ae>%n%b" output; anything not done and why.
Then stop and wait.
```

You check

- `npm run build` then `npm run preview`, open `http://localhost:4173` in Chrome: an install icon appears in the address bar.
- Chrome DevTools → Application → Service Workers → tick Offline, reload `/dashboard`: it still opens.

S08 · Quality pass

Essential · Replaces C13

Goal. Make every screen robust and accessible on phones and laptops, shrink the main bundle, and fix the test failure that happens when the suite starts from a lowercase drive path.

```
PROMPT S08 - Quality pass: code splitting, error boundaries, accessibility,
layout, drive-letter test bug

You are working on FollowThru (Fynd Hackathon 2026, challenge C14,
"Chase what's late. Never what's done.") in D:\followthru.
Single-laptop workflow: one developer, one branch, no Track A / Track B split.

BEFORE EDITING
1. Read CLAUDE.md, README.md, docs/integration-status.md and
   docs/fixture-notes.md.
2. Run: git status --short ; git branch --show-current ; git log --oneline -8
3. Work only on branch full-integration (git checkout full-integration ;
   git pull origin full-integration). If git status shows changes you did
   not make in this session, STOP and show them. Never stash, reset or
   discard. Leave docs/TRACK_A_BACKEND_PROMPT.md untracked and untouched.
4. Run npm test once. If it fails before you change anything, report first.

PROJECT FACTS AND RULES
All facts and rules in CLAUDE.md apply. In short: src/engine/types.ts is the
single contract (additive changes only); src/engine is pure; UI uses
src/store hooks only; every write is audit-logged; shadow mode only; never
invent labels, fields, accuracy or scores; synthetic data is labelled.

GIT AND REPORTING RULES
- test + typecheck + build pass before every commit; review
  git diff --staged; stage by explicit path, never git add -A or git add .
- Commits: lowercase, 3-8 words, no symbols, no prefixes, one line. No
  Co-Authored-By and no AI or Claude mention anywhere. Never change git
  config or use --author.
- No node_modules, dist, .env, keys, keystores, APKs or personal data.
- No force push, reset --hard, clean, checkout -- ., rebase of pushed
  branches or branch deletion. Never merge into main without the exact
  phrase: merge into main. Push after each commit:
  git push origin full-integration
- Never claim results you did not produce; never weaken tests.

CURRENT CONTEXT (verified at cad765d; re-check against the code)
Main JS chunk 658.69 kB (209.19 kB gzip); 3D already lazy (SceneCanvas
chunk). No route-level code splitting, no error boundaries, no automated
accessibility audit. e2e checks horizontal overflow only on the dashboard at
360 px. Known bug: Vitest started from d:/followthru (lowercase drive) fails
every suite with "TypeError: Cannot read properties of undefined (reading
'config')" and "no tests"; from D:/followthru all tests pass. VS Code
sometimes starts in the lowercase path.

TASK: fix, do not redesign.
1. Route-level code splitting with React.lazy and Suspense for every page
   except Landing (tests/app.test.tsx asserts the landing synchronously;
   switch other assertions to findBy where needed). Keep EngineProvider at
   the top of App so store state survives navigation. Report main chunk
   size before and after (target below 150 kB gzip).
```

```
2. src/components/RouteErrorBoundary.tsx around the routes: friendly
   message, Reload, and Reset local data (confirm Dialog, then
   actions.resetAll). Test with a component that throws.
3. Accessibility: add @axe-core/playwright (dev dependency) and
   e2e/a11y.spec.ts that loads the demo as Priya and checks landing, setup,
   profiles, dashboard, commitments, a detail page, queue, flows, team and
   evidence: zero serious or critical violations. Fix labels, contrast,
   landmarks, button names and heading order. Status never relies on colour
   alone.
4. Responsive sweep in e2e: widths 360, 390, 768, 1024, 1440 over the main
   pages with no horizontal overflow; fix layouts that fail.
5. Reduced motion: an e2e test with reducedMotion 'reduce' confirms the
   landing renders Fallback2D and nothing animates.
6. Drive-letter bug: reproduce first with
   node -e "require('child_process').spawnSync('npx vitest run',
   {cwd:'d:/followthru',shell:true,stdio:'inherit'})"
   (all suites fail) and the same with cwd 'D:/followthru' (all pass). Find
   the root cause in how Vitest resolves the config and the projects
   (engine, ui) in vitest.config.ts when drive letter case differs; check
   the Vitest changelog or issues instead of guessing. Fix it so npm test
   passes from both paths (for example by normalising the root). Add a check
   script (for example scripts/check-drive-case.cjs run by an npm script)
   that runs the suite with a lowercase cwd. Record cause and fix in
   docs/integration-status.md. Never hide the failure by skipping tests.
7. Remove dead code, console.log calls and TODOs.

TESTING
npm test (run it 10 times in a loop), npm run typecheck, npm run build
(paste chunk sizes), npm run e2e including the new specs, the drive-case
check script.

COMMITS (push after each)
split routes into lazy chunks / add route error boundary / fix
accessibility issues / fix layouts at small widths / run tests from any
drive letter case

ACCEPTANCE CRITERIA
- Main chunk smaller than before, sizes reported.
- axe: zero serious or critical violations on all listed pages.
- No horizontal overflow at the five widths.
- npm test passes from both d:/followthru and D:/followthru; all suites
  pass.

FINAL REPORT
Files changed; real results of every command (10-run loop results, typecheck,
chunk sizes before and after, e2e, a11y counts); drive-letter root cause in
two or three lines; commits with hashes;
git log -6 --format="%an <%ae>%n%b" output; anything not done and why.
Then stop and wait.
```

You check

- `npm run dev -- --host`, open the Network URL on your phone, click through as coordinator and member.
- Read the chunk sizes and the drive-letter fix in Claude’s report.

S09 · Commitment detail explainability and landing story

Part 1 Recommended · Part 2 Optional (cut first) · Replaces C14 + C15

Goal. Explain each decision in plain sentences and let a viewer scrub through time to watch owner, deadline, status and draft change. Optionally, replace the minimal hero with the planned scroll story using real counts and a complete fallback.

Short on time? Delete Part 2 from the block before pasting. If you skip both parts, the rest of the playbook still works.

```
PROMPT S09 - Commitment detail explainability and landing story

You are working on FollowThru (Fynd Hackathon 2026, challenge C14,
"Chase what's late. Never what's done.") in D:\followthru.
Single-laptop workflow: one developer, one branch, no Track A / Track B split.

BEFORE EDITING
1. Read CLAUDE.md, README.md, docs/integration-status.md and
   docs/fixture-notes.md.
2. Run: git status --short ; git branch --show-current ; git log --oneline -8
3. Work only on branch full-integration (git checkout full-integration ;
   git pull origin full-integration). If git status shows changes you did
   not make in this session, STOP and show them. Never stash, reset or
   discard. Leave docs/TRACK_A_BACKEND_PROMPT.md untracked and untouched.
4. Run npm test once. If it fails before you change anything, report first.

PROJECT FACTS AND RULES
All facts and rules in CLAUDE.md apply. In short: src/engine/types.ts is the
single contract (additive changes only); src/engine is pure; UI uses
src/store hooks only; every write is audit-logged; shadow mode only; never
invent labels, fields, accuracy or scores; synthetic data is labelled.

GIT AND REPORTING RULES
- test + typecheck + build pass before every commit; review
  git diff --staged; stage by explicit path, never git add -A or git add .
- Commits: lowercase, 3-8 words, no symbols, no prefixes, one line. No
  Co-Authored-By and no AI or Claude mention anywhere. Never change git
  config or use --author.
- No node_modules, dist, .env, keys, keystores, APKs or personal data.
- No force push, reset --hard, clean, checkout -- ., rebase of pushed
  branches or branch deletion. Never merge into main without the exact
  phrase: merge into main. Push after each commit:
  git push origin full-integration
- Never claim results you did not produce; never weaken tests.

CURRENT CONTEXT (verified at cad765d; re-check against the code)
CommitmentDetail.tsx shows facts, an evidence timeline (event type, time,
person, message id, thread, rule, quote), linked messages, submissions,
blockers, the draft card and role actions; no plain-English reasons, time
travel or full-thread view. Landing.tsx shows a headline, subtitle, Open app
link to /setup and <Stage> (src/three/Stage.tsx) which lazy-loads
SceneCanvas.tsx (one wireframe mesh) and falls back to Fallback2D.tsx
without WebGL or with reduced motion. Lenis is installed but unused. Tokens
in src/styles/tokens.ts and global.css.

PART 1 - EXPLAINABILITY (src/engine stays unchanged)
1. src/components/domain/explain.ts, pure: explain(commitment, draft,
```

```
   people, threads) returns sentences with message ids and rule names, e.g.
   "Owner is Lena because Amir handed it over (m_07) and Lena accepted
   (m_08)." and "No reminder: completed (m_10, linked from another thread
   by keyword overlap)." Unit tests for each event kind in
   tests/explain.test.ts.
2. Time travel: slider over the sorted unique times of the commitment's
   linked messages, decisions and the evaluation clock. For position t call
   runEngine with messages sentAt <= t, decisions at <= t, manual tasks
   createdAt <= t, now = t and no previous drafts (say on screen that
   drafts here are recomputed, not the stored queue). Memoise per position,
   debounce 150 ms. Show status, owner, deadline and draft state at t. Step
   buttons always; a play button disabled when reduced motion is on. Helper
   in src/components/domain/timeTravel.ts. Never writes to the store.
3. Thread view: clicking a message id opens the Dialog (sheet variant) with
   the whole thread in order, that message highlighted, times in the
   author's and the viewer's timezone.
4. Tests: explain unit tests; time travel on demo commitment c_m_03 (open
   before m_10, done at m_10); UI test that moving the slider changes the
   status chip; an e2e step that uses the slider.
   Commits: add plain english why panel / add time travel slider / add
   thread view sheet

PART 2 - LANDING SCROLL STORY (optional; original code and assets only)
5. One pinned canvas behind HTML sections: hero ("Chase what's late. /
   Never what's done."), chaos (message chips tangled), scan (commitments
   light up), sorted (chips orbit in status colours: done mint, overdue
   coral, tentative amber, deferred sky), queue (overdue chips stack into
   cards, done chips fade with a suppressed pulse), edge cases (six cards:
   maybe Friday, timezones, delegation, completion elsewhere, silence versus
   not done, deadline moved), CTA (Open app, View evidence).
6. Instanced mesh: at most 320 chips on desktop, 120 on phones; per-section
   targets precomputed; useFrame lerps positions (no React state per
   frame); pause when the tab is hidden; frameloop demand when idle. Create
   src/three/StoryScene.tsx and small helpers as needed.
7. Counts come from the real engine via useEngine when data is loaded,
   labelled with the dataset name ("synthetic" for the demo); without data
   show no numbers.
8. Lenis only on the landing page, off with reduced motion. Fallback2D and
   CSS-only sections when WebGL is missing or motion is reduced.
9. Open app goes to /dashboard when data and a profile exist, otherwise
   /setup. Only the landing route changes.
10. Tests: tests/app.test.tsx landing assertions still pass; e2e checks no
    console errors and the reduced-motion fallback. Keep 3D in a lazy chunk
    and report its size.
    Commits: add landing story sections / add 3d story scene / add landing
    fallback

TESTING
After each part: npm test, npm run typecheck, npm run build (report 3D chunk
size after Part 2), npm run e2e.

ACCEPTANCE CRITERIA
- Every commitment shows plain-English reasons with message ids; the slider
  shows the deadline-to-done story for c_m_03; thread sheet opens; engine
  untouched.
- (Part 2) Story renders smoothly on a laptop and degrades to the fallback
  with reduced motion; counts match the dashboard; no console errors.
- All suites pass.

FINAL REPORT
```

```
Files changed; real results of every command (test counts, typecheck, build
and chunk sizes, e2e); commits with hashes;
git log -6 --format="%an <%ae>%n%b" output; whether Part 2 was run;
anything not done and why. Then stop and wait.
```

You check

- Open `/commitments/c_m_03`, drag the slider from start to end: status goes from open to done. Click a message id and read the thread sheet.
- (Part 2) Scroll the landing page on laptop and phone; turn on reduced motion in your OS and confirm the static version appears.

S10 · Optional AI-assisted extraction with a labelled rules fallback

Optional (needs your own API key) · Replaces C16

Goal. Add an opt-in AI pre-pass that can only add suggestions or review flags, never decide status, with an honest fallback label.

```
PROMPT S10 - Optional AI-assisted extraction with a labelled rules fallback

You are working on FollowThru (Fynd Hackathon 2026, challenge C14,
"Chase what's late. Never what's done.") in D:\followthru.
Single-laptop workflow: one developer, one branch, no Track A / Track B split.

BEFORE EDITING
1. Read CLAUDE.md, README.md, docs/integration-status.md and
   docs/fixture-notes.md.
2. Run: git status --short ; git branch --show-current ; git log --oneline -8
3. Work only on branch full-integration (git checkout full-integration ;
   git pull origin full-integration). If git status shows changes you did
   not make in this session, STOP and show them. Never stash, reset or
   discard. Leave docs/TRACK_A_BACKEND_PROMPT.md untracked and untouched.
4. Run npm test once. If it fails before you change anything, report first.

PROJECT FACTS AND RULES
All facts and rules in CLAUDE.md apply. In short: src/engine/types.ts is the
single contract (additive changes only, contract test updated on purpose,
changes logged); src/engine is pure - all network work happens in the store
before runEngine; UI uses src/store hooks only; every write is audit-logged;
silence never changes status; ambiguous evidence -> needsReview; shadow
mode only; never invent labels, fields, accuracy or scores.

GIT AND REPORTING RULES
- test + typecheck + build pass before every commit; review
  git diff --staged; stage by explicit path, never git add -A or git add .
- Commits: lowercase, 3-8 words, no symbols, no prefixes, one line. No
  Co-Authored-By and no AI or Claude mention in commit messages or PR text.
  Never change git config or use --author.
- Never commit an API key, .env, keystores, APKs or personal data.
- No force push, reset --hard, clean, checkout -- ., rebase of pushed
  branches or branch deletion. Never merge into main without the exact
  phrase: merge into main. Push after each commit:
  git push origin full-integration
- Never claim results you did not produce; never weaken tests.

CURRENT CONTEXT (verified at cad765d; re-check against the code)
EngineFlags.ai exists; modeLabel in src/engine/run.ts returns
'Rules (fallback)' when ai is true because no provider exists. The Data &
settings tab explains this. S03 added src/engine/extractor.ts (Extractor
interface, rulesExtractor). S06 added exportBackup.

TASK: optional Gemini-assisted signal suggestions, OFF by default.
1. Check the current Gemini API documentation for the endpoint and a
   current free-tier flash model name; make the model a setting. Do not
   guess names from memory.
2. The key is entered on the Data & settings tab, stored only in IndexedDB
   settings, excluded from backups, exports, audit entries and the API, and
   never committed. Never ask me to paste it into this chat.
3. AI pre-pass in the browser (src/store/ai.ts, async), per thread, asking
```

```
   for JSON only, validated with Zod: [{ messageId, kind, hedged,
   refersToMessageId?, confidence }]. Invalid JSON -> fallback.
4. Deterministic, pure merge rule: EngineInput gains optional aiSignals:
   Record<string, Signal[]> (additive). The engine accepts an AI suggestion
   only when the rules agree; disagreement or confidence below 0.7 adds
   review reason 'ai_disagreement' (additive ReviewReason) and never changes
   status. Deadlines still go through parseDeadline.
5. Fallback with a visible reason for: no key, offline, timeout (8 s),
   HTTP 429, error, invalid output -> mode "Rules (fallback)". Success ->
   "AI assisted". Show the mode in the TopBar and on the Evidence page.
6. Official evidence (npm run evidence) always runs rules-only; say so in
   the scorecard header.
7. Tests (tests/ai.test.ts, mocked fetch): valid output merges, invalid
   output falls back, timeout falls back, low confidence goes to review, the
   key never appears in exportBackup output.

FILES
Create: src/store/ai.ts, tests/ai.test.ts. Modify: src/engine/types.ts,
src/engine/run.ts, src/engine/history.ts, src/store/EngineProvider.tsx,
src/pages/Evidence.tsx, src/components/shell/TopBar.tsx,
scripts/evidence.ts, tests/contract.test.ts, docs/integration-status.md.

TESTING
npm test, npm run typecheck, npm run build, npm run e2e (must pass without a
key).

COMMITS (push after each)
add optional ai suggestions / add ai fallback labels / add ai tests

ACCEPTANCE CRITERIA
- With no key the TopBar shows Rules and everything works.
- With a key, suggestions never change status on their own; fallbacks are
  labelled.
- The key never leaves IndexedDB; tests prove it is excluded from backups.

FINAL REPORT
Files changed; real results of every command; the model name and the doc
page you took it from; commits with hashes;
git log -4 --format="%an <%ae>%n%b" output; anything not done and why.
Then stop and wait.
```

You check

- Without a key, the top bar says Rules.
- Optional: add your own key in the app, reload, and check the mode label and the review reasons.

S11 · Screenshot evidence and project documentation

Essential · Replaces C17

Goal. Produce timestamped screenshots as evidence and the documents a judge reads, with honest placeholders for anything only you can supply.

```
PROMPT S11 - Screenshot evidence and project documentation

You are working on FollowThru (Fynd Hackathon 2026, challenge C14,
"Chase what's late. Never what's done.") in D:\followthru.
Single-laptop workflow: one developer, one branch, no Track A / Track B split.

BEFORE EDITING
1. Read CLAUDE.md, README.md, docs/integration-status.md and
   docs/fixture-notes.md.
2. Run: git status --short ; git branch --show-current ; git log --oneline -8
3. Work only on branch full-integration (git checkout full-integration ;
   git pull origin full-integration). If git status shows changes you did
   not make in this session, STOP and show them. Never stash, reset or
   discard. Leave docs/TRACK_A_BACKEND_PROMPT.md untracked and untouched.
4. Run npm test once. If it fails before you change anything, report first.

PROJECT FACTS AND RULES
All facts and rules in CLAUDE.md apply. Screenshots come from the real
running app with real engine output; nothing is mocked or edited. Never
invent results, names, URLs, scores or experiment rows.

GIT AND REPORTING RULES
- test + typecheck + build pass before every commit; review
  git diff --staged; stage by explicit path, never git add -A or git add .
- Commits: lowercase, 3-8 words, no symbols, no prefixes, one line. No
  Co-Authored-By and no AI or Claude mention anywhere, including docs.
  Never change git config or use --author.
- No node_modules, dist, .env, keys, keystores, APKs or personal data.
- No force push, reset --hard, clean, checkout -- ., rebase of pushed
  branches or branch deletion. Never merge into main without the exact
  phrase: merge into main. Push after each commit:
  git push origin full-integration
- Never claim results you did not produce; never weaken tests.

CURRENT CONTEXT (verified at cad765d; re-check against the code)
README.md has run commands, data sources and an architecture table. docs/
has integration-status.md, fixture-notes.md, track-a-status.md,
track-b-status.md and planning documents. e2e/app.spec.ts saves four
screenshots to test-results/screens/ (git-ignored). No screenshots script,
no evidence/screenshots folder, no architecture, pilot plan, growth
experiment or business case documents. docs/api.md and
docs/product-change.md exist if S04 and S06 ran.

TASK
1. scripts/screenshots.ts using @playwright/test chromium with channel
   'chrome' (installed Chrome, no download). Build, start npm run preview on
   port 4173 or reuse a running server. Load the demo, or the challenge
   fixture when data/fixture.json exists. Capture at 1440x900 and 390x844:
   landing, dashboard, commitments, detail of a completed-in-another-thread
   commitment (demo c_m_03), queue with recipient review, suppressed lane,
   flows, team as coordinator, member workspace as Lena, evidence
```

```
   scorecard, audit log. Save to evidence/screenshots/<timestamp>/ with
   index.md listing file, time, page, what it proves, and data source
   (SYNTHETIC or challenge fixture). Add script "screenshots". Commit only
   the latest set.
2. README.md rewrite: banner screenshot, tagline, LIVE_URL, APK_URL and
   DEMO_VIDEO_URL as clearly marked placeholders, the problem in three
   lines, features, edge cases table (case -> rule in code -> test file ->
   evidence), scoring table (requirement -> where -> proof file),
   architecture diagram in Mermaid, stack with one-line reasons, data flow,
   folder structure, quick start with the real scripts, API docs link,
   product change summary, honest limitations (synthetic demo data, fixture
   status, shadow mode only, local demo profiles are not security, not
   production ready), roadmap, team credits with TEAMMATE_1 and TEAMMATE_2
   placeholders (never invent names).
3. docs/architecture.md: modules, pure engine design, data model,
   commitment state machine (Mermaid), idempotency and versioning,
   freshness check, timezone strategy, why IndexedDB instead of an external
   database.
4. docs/pilot-plan.md: shadow a coordinator; metrics false reminder rate,
   overdue-action coverage, approval rate with formulas; thresholds before
   controlled sends (false reminders under 5 percent, coverage at least 90
   percent, approval at least 80 percent over two weeks); go or no-go
   table; coordinator feedback template.
5. docs/growth-experiment.md: table for ten overdue commitments with
   columns id, who was approached (role), date, channel, draft shown,
   response, acted within 48 h, useful (definition), notes, plus Observed
   versus Assumption columns. Leave every row EMPTY for me.
6. docs/business-case.md: a short placeholder saying where I will add the
   business case PDF and pitch deck; no business results.

TESTING
npm run screenshots (open several images to confirm), npm test,
npm run typecheck, npm run build. Check the Mermaid blocks render on GitHub.

COMMITS (push after each)
add screenshot script / save screenshot evidence / update readme for judges
/ add architecture and pilot docs / add growth experiment template

ACCEPTANCE CRITERIA
- evidence/screenshots/<timestamp>/ with index.md and both sizes.
- README and docs are factual, with placeholders clearly marked.
- No invented results, names or URLs anywhere.

FINAL REPORT
Files changed; real results of every command; the screenshot folder path
and file count; commits with hashes; git log -5 --format="%an <%ae>%n%b"
output; anything not done and why. Then stop and wait.
```

You check

- Open `evidence/screenshots/<timestamp>/index.md` and a few images.
- Read `README.md` on GitHub; search it for `LIVE_URL` to see the placeholders.
- Fill `docs/growth-experiment.md` with your real rows when you have them.

S12 · Final QA and release check

Essential · Re-runnable before every merge, deploy or submission · Replaces C18 + RELEASE CHECK

Goal. Verify the whole brief end to end, report PASS, FAIL or BLOCKED with proof paths, and fix only small issues. The first run creates `docs/release-check.md`; later runs update it.

```
PROMPT S12 - Final QA and release check

You are working on FollowThru (Fynd Hackathon 2026, challenge C14,
"Chase what's late. Never what's done.") in D:\followthru.
Single-laptop workflow: one developer, one branch, no Track A / Track B split.

BEFORE EDITING
1. Read CLAUDE.md, README.md, docs/integration-status.md,
   docs/fixture-notes.md and, if present, docs/release-check.md and
   docs/product-change.md.
2. Run: git status --short ; git branch --show-current ; git log --oneline -8
3. Work only on branch full-integration. A clean tree is required: if
   git status is not clean, show it and ask me. Never stash, reset or
   discard. Leave docs/TRACK_A_BACKEND_PROMPT.md untracked and untouched.

PROJECT FACTS AND RULES
All facts and rules in CLAUDE.md apply. No feature work in this prompt: it
verifies and documents. Report first, then fix only trivial issues; anything
larger is reported as FAIL with a proposed FIX prompt (S14).

GIT AND REPORTING RULES
- test + typecheck + build pass before every commit; review
  git diff --staged; stage by explicit path, never git add -A or git add .
- Commits: lowercase, 3-8 words, no symbols, no prefixes, one line. No
  Co-Authored-By and no AI or Claude mention anywhere. Never change git
  config or use --author.
- No force push, reset --hard, clean, checkout -- ., rebase of pushed
  branches or branch deletion. Never merge into main without the exact
  phrase: merge into main. Push after each commit:
  git push origin full-integration
- Never claim results you did not produce; never weaken tests.

TASK
1. Run every script that exists in package.json: npm test (twice),
   npm run typecheck, npm run build, npm run e2e, npm run e2e:pwa (if
   present), npm run evidence -- --label release (fixture if present; else
   --demo, labelled SYNTHETIC, with the scoring line BLOCKED),
   npm run screenshots (if present).
2. Write or update docs/release-check.md (with today's date) with PASS, FAIL
   or BLOCKED and a proof path for each item:
   - ten labelled commitments correct with evidence ids (BLOCKED without
     the fixture);
   - zero active reminders for completed, cancelled or deferred work;
   - one action one draft; a deadline update changes the same draft;
   - maybe Friday tentative, timezones, delegated owner, completion in
     another thread, no response versus not completed;
   - silence never changes status (response pending shown);
   - stale-message suppression (completion after drafting) and recipient
     review blocking approval until confirmed;
   - starting state, negotiated scope and final evidence saved;
```

```
   - validation messages for bad input; JSON and CSV import and export;
   - readable evidence: scorecard, screenshots, test report, timestamps,
     hashes;
   - optional AI labelled and rules fallback with no key (or AI absent);
   - coordinator and member flows with permissions; audit log;
   - PWA installable, offline reload, phone layout;
   - API health, process, evidence;
   - README honest limitations; growth experiment and pilot plan present
     (rows may be empty until I fill them).
3. Hygiene: git grep for secrets (key=, apiKey, AIza, BEGIN PRIVATE KEY,
   .env); confirm .gitignore covers .env*, *.keystore, *.jks, *.apk, *.aab,
   signing-key-info.txt; no console.log or TODO in src; run
   git log --format="%an <%ae>%n%B" and confirm there are no Co-Authored-By
   lines or AI mentions on this branch and every author is me;
   docs/TRACK_A_BACKEND_PROMPT.md still untracked and untouched.
4. Commit the release evidence run (or, if synthetic and ignored, copy its
   scorecard summary into release-check.md) and docs/release-check.md.
   Fix only trivial issues found here.

COMMITS (push after each)
save release evidence / add release check (later runs: update release
check) / remove unused files (only if any)

ACCEPTANCE CRITERIA
- docs/release-check.md exists; every item has PASS, FAIL or BLOCKED with
  proof.
- All scripts pass, or failures are reported honestly with next steps.
- No secrets, keystores or APKs tracked; no co-author or AI mentions in
  commits.

FINAL REPORT
Files changed; real results of every command; the full PASS/FAIL/BLOCKED
table; every BLOCKED line names what is missing; commits with hashes.
Do not merge into main. Then stop and wait.
```

You check

- Read `docs/release-check.md`. Every BLOCKED line should name what is missing (for example the fixture).
- Decide whether to proceed to S13.

S13 · Merge, deploy on Vercel and Android APK

Essential (needs your authorisation and Vercel account) · APK optional or as your submission requires · Replaces C19 + C20

Goal. Prepare the Android-safe repo settings, open a pull request from `full-integration`, merge only after your explicit go-ahead, verify the live Vercel URL, add Digital Asset Links, and publish the APK as a GitHub release — never committing signing keys. This prompt runs as a single chat that waits for you between parts.

```
PROMPT S13 - Merge, deploy on Vercel and Android APK

You are working on FollowThru (Fynd Hackathon 2026, challenge C14,
"Chase what's late. Never what's done.") in D:\followthru.
Single-laptop workflow: one developer, one branch, no Track A / Track B split.

BEFORE EDITING
1. Read CLAUDE.md, README.md, docs/integration-status.md and
   docs/release-check.md.
2. Run: git status --short ; git branch --show-current ; git log --oneline -8
3. Work only on branch full-integration. If git status shows changes you did
   not make in this session, STOP and show them. Never stash, reset or
   discard. Leave docs/TRACK_A_BACKEND_PROMPT.md untracked and untouched.

RULES
- All facts and rules in CLAUDE.md apply.
- test + typecheck + build pass before every commit; stage by explicit
  path. Commits and PR titles: lowercase, 3-8 words, no symbols. PR bodies:
  plain summary, no AI or Claude mention, no co-author lines, no
  "Generated with" footer. Never change git config or use --author.
- Never force push, rebase, reset --hard, clean or delete branches.
- Merge into main ONLY after I reply exactly: merge into main.
- Publish a release ONLY after I reply exactly: publish release.
- Never read, copy or commit signing.keystore, *.jks or
  signing-key-info.txt. Never invent a URL, release or result. If a
  verification fails, report it and change nothing.

CURRENT CONTEXT (verified at cad765d; re-check)
GitHub main = 3fd93f3; full-integration contains all of it. PRs #1-#4 were
merged with merge commits. vercel.json sets the build command, output dist
and the SPA rewrite, which would also catch /.well-known/assetlinks.json
unless excluded. The demo option appears in production builds only when
VITE_DEMO_MODE=true. S07 provides PNG icons, manifest and service worker.

PART A - PREPARE (you may do this now)
1. Android safety: add to .gitignore *.keystore, *.jks,
   signing-key-info.txt, *.apk, *.aab and pwabuilder-output/. Commit
   "ignore android signing files".
2. Update vercel.json so /.well-known/(.*) is NOT rewritten to index.html
   and is served with Content-Type application/json. Add a test or a
   documented curl check against npm run preview. Commit
   "serve asset links file". Push.
3. Confirm docs/release-check.md exists, npm test, typecheck, build and e2e
   pass, and local and origin/full-integration are at the same commit.
4. git fetch origin; confirm git merge-base --is-ancestor origin/main
   full-integration. If not, STOP and report; do not rebase.
5. gh pr create --base main --head full-integration with a lowercase title
```

```
   without symbols (for example "complete followthru app") and a plain body
   linking the release check. Print the PR URL and STOP.

PART B - only when I reply exactly: merge into main
6. Re-run npm test and npm run build, then
   gh pr merge <number> --merge (merge commit, no squash, no force, keep
   the branch). Verify origin/main equals the merge commit and that a fresh
   checkout of main builds: git worktree add in a temporary folder outside
   the repo, npm ci, npm run build, then remove that worktree. Report and
   STOP. I will now deploy on Vercel.

PART C - after I paste the Vercel URL
7. Verify it: fetch /api/health (expect ok true), /, /manifest.webmanifest,
   and run the e2e smoke journey against it if practical (Playwright base
   URL override). Report real results.
8. Replace LIVE_URL in README.md and docs/api.md with the URL, commit
   "add live demo link", push, open a small PR to main, and merge it only
   after I write merge into main again. Then STOP. I will now use
   PWABuilder.

PART D - after I paste the assetlinks.json content from the PWABuilder zip
9. Create public/.well-known/assetlinks.json with exactly that content
   (validate it is JSON). Commit "add asset links", push, PR to main, merge
   only after merge into main. After the deploy, fetch
   <LIVE_URL>/.well-known/assetlinks.json and confirm it returns that JSON.

PART E - after I give the local .apk path and write: publish release
10. gh release create v1.0.0 <path-to-apk> --title "FollowThru 1.0.0"
    --notes "<plain notes, no AI mention>" --target main. Print the release
    URL from gh's output.
11. Replace APK_URL in README.md with the release download URL, commit
    "add apk download link", push, PR to main as above.

ACCEPTANCE CRITERIA
- PR created and merged only after the exact phrase; main builds from a
  fresh checkout.
- Live URL verified (health, landing, manifest) and recorded in README.
- Keystore and signing info never tracked (git ls-files shows none).
- assetlinks.json served as JSON from the live site.
- GitHub release exists with the APK; README links to it.

REPORT AFTER EVERY PART
Files changed, real command output, commits with hashes, PR or release
URLs exactly as printed, and what you are waiting for from me. Then stop.
```

You do between parts

- **Vercel (after Part B):** vercel.com → sign in with GitHub → Add New → Project → import `shivv-14/followthru`. Preset Vite, build `npm run build`, output `dist`, production branch `main`. Set `VITE_DEMO_MODE=true` if the fixture is not committed. Deploy, open `<your-url>/api/health`, paste the URL into the chat.
- **PWABuilder (after Part C):** pwabuilder.com → enter the live URL → Start → fix red items → Package for stores → Android → Generate. Unzip outside the repo (for example `D:\followthru-android\`). Keep `signing.keystore` and `signing-key-info.txt` safe, never in the repo. Paste the `assetlinks.json` content into the chat.
- **APK (Part E):** give the APK path and write `publish release`. Install from the release link on an Android phone: no browser bar once asset links are live.

5. Operational prompts

S14 · Resume, fix or blocker report

Operational · Replaces CONTINUE + FIX + BLOCKER REPORT

Use: after a usage limit or crash (RESUME), whenever something breaks (FIX), or when you are waiting on files, accounts or decisions (BLOCKERS). Keep one MODE line and fill in its angle brackets.

```
PROMPT S14 - Resume, fix or blocker report

MODE: <RESUME | FIX | BLOCKERS>
LAST PROMPT I RAN (RESUME): <S01 ... S13>
PROBLEM (FIX): <one or two lines describing what you saw>
ERROR (FIX): <exact text or screenshot description>
WHERE (FIX): <command or page, e.g. npm test, /queue on phone, profile Lena>

You are working on FollowThru (Fynd Hackathon 2026, challenge C14) in
D:\followthru. Single laptop, branch full-integration only.

ALWAYS FIRST
1. Read CLAUDE.md, README.md, docs/integration-status.md and, if present,
   docs/release-check.md and docs/product-change.md.
2. Run: git status --short ; git branch --show-current ;
   git log --oneline -15 ; git fetch origin ; git status -sb
3. If git status shows uncommitted changes you did not make, list them and
   ask me before touching them. Never stash, reset or discard. Leave
   docs/TRACK_A_BACKEND_PROMPT.md untracked.

RULES
- Keep the engine pure and the contract additive. Never delete, skip or
  weaken a test to make it pass.
- Commits: lowercase, 3-8 words, no symbols, one line, no co-author lines,
  no AI mention; stage by explicit path; test + typecheck + build before
  each commit; push to origin full-integration.
- No force push or destructive git. Never merge into main unless I write
  exactly: merge into main. Never invent inputs, URLs or results.

IF MODE = RESUME
4. We follow the FollowThru Continuation Prompt Playbook (15 prompts). Check
   that the acceptance criteria of the LAST PROMPT are met by inspecting
   the code and running npm test, npm run typecheck and npm run build (and
   npm run e2e if it touched the UI).
5. Report what is complete, what is missing and what failed, with real
   output. If something from that prompt is missing, finish only that part
   with small commits, push, then wait for my next prompt.

IF MODE = FIX
4. Reproduce it first (a failing test, a script run or a Playwright step).
   If you cannot reproduce it, say so and show what you tried.
5. Find the root cause and explain it in two or three lines naming the file
   and function (for example src/engine/history.ts findTarget).
6. Fix the cause, not the symptom. Add or update a test that fails before
   the fix and passes after.
7. Run npm test, npm run typecheck, npm run build, and npm run e2e if the
   UI is involved. Review git diff --staged, commit with a short lowercase
```

```
   message such as "fix stale draft check before approval", push, and
   report the before and after output.

IF MODE = BLOCKERS (report only; do not invent any missing input)
4. Check each external dependency: data/fixture.json present? committed?;
   official deadline and demo format confirmed?; Vercel project and live URL
   (LIVE_URL placeholders)?; PWABuilder output, assetlinks.json, APK release
   (APK_URL)?; optional Gemini key (never ask me to paste it; you cannot see
   it unless I tell you); business case PDF, pitch deck, demo video
   (DEMO_VIDEO_URL), teammate names (TEAMMATE_1, TEAMMATE_2), real
   growth-experiment rows?
5. For each missing item: what is blocked, who can provide it (me or the
   organisers) and the exact action needed.
6. List the playbook prompts S01 to S13 that can still run without these
   inputs, in dependency order, and the first to run next. Continue with
   it only if I reply go.

FINAL REPORT
Real command output, commits with hashes (if any), and the next step.
Then stop and wait.
```

S15 · Scorecard and integration check

Operational · Replaces SCORECARD + INTEGRATION CHECK

Use: any time you want the real numbers, after UI work, or before a demo. No code changes unless you approve the findings.

```
PROMPT S15 - Scorecard and integration check (read only)

You are working on FollowThru (Fynd Hackathon 2026, challenge C14) in
D:\followthru, branch full-integration. Read CLAUDE.md first. Do not change
any code in this prompt; propose fixes and wait for my go.

PART 1 - SCORECARD
1. If data/fixture.json exists run npm run evidence -- --label check;
   otherwise npm run evidence -- --demo and state clearly that the data is
   SYNTHETIC and the owner/deadline/status check is not scored.
2. Open the newest scorecard.md in evidence/ and report exactly:
   - labelled commitments correct: x of y (or "not scored: no labels");
   - active reminders on completed, cancelled or deferred work: n;
   - duplicate drafts: n; re-run identical: yes or no;
   - each failing labelled item with expected versus actual and evidence
     message ids;
   - the evidence folder path and its manifest hashes.
3. If anything fails, diagnose the likely cause (file and rule) and propose
   the smallest general fix.

PART 2 - INTEGRATION CHECK
4. List every route in src/App.tsx and, for each page in src/pages, the data
   it reads (useEngine fields, store hooks) and the actions it calls
   (useActions functions). Flag any hardcoded arrays of domain data, any
   import from src/mocks other than demoDataset.json in Setup, any button
   without a handler, any "coming soon" or placeholder text, and any
   component importing Dexie or src/store/db.ts directly.
5. For each action in src/store/actions.ts confirm: permission check, repo
   write, audit entry, and a test that covers it (grep tests/).
6. grep src for TODO, FIXME, console.log, Math.random used for data, and
```

```
   Date.now inside src/engine.
7. Run npm test and npm run e2e and report results.

OUTPUT
- The scorecard numbers from Part 1.
- A table: screen or action, data source, real or placeholder, test
  coverage, issue.
- Proposed fixes as S14 FIX prompts, ordered by impact.
Then stop and wait for my go before changing anything.
```

6. What Claude does and what you do

Action	Claude (locally)	You
Code, tests, docs, evidence runs, screenshots	Yes, S01–S12	Run the “You check” steps
Copy the challenge fixture into `data/`	No	Yes, before S02
Commit the fixture	Only after you say yes	Decide (check the challenge terms)
Pull request to main	Creates it (S13 Part A)	Review it
Merge into main	Only after the exact phrase `merge into main`	Authorise
Vercel project and deploy	No	Import repo, preset Vite, output dist, set `VITE_DEMO_MODE=true` if the fixture is not committed
Verify the live URL and record it	Yes, after you paste it (S13 Part C)	Paste the real URL
PWABuilder package	No	Generate on pwabuilder.com; keep the keystore outside the repo
assetlinks.json	Adds the file you paste (S13 Part D)	Paste its content
GitHub release with the APK	Only after `publish release` and the APK path (S13 Part E)	Provide the path and approve
Gemini key	Never stores it in files	Enter it in the app only if you run S10
Growth experiment, business case, pitch deck, demo video, team names	Templates and placeholders only	Real content

Claude must never fabricate a deployed URL, an APK, a release, a Lighthouse result, a score or a submission. Placeholders `LIVE_URL`, `APK_URL`, `DEMO_VIDEO_URL`, `TEAMMATE_1` and `TEAMMATE_2` stay until you supply real values. Never commit `signing.keystore`, `signing-key-info.txt`, `.env` files or keys.

7. Final release and submission checklist

Item	Where to prove it	Produced by or blocked on	Done?
Live link opens on phone and laptop	README LIVE_URL, S13 Part C report	Your Vercel account	☐
APK installs on Android	GitHub release (S13 Part E)	PWABuilder and your phone	☐

Item	Where to prove it	Produced by or blocked on	Done?
Scorecard: labelled commitments correct, 0 wrong reminders, 0 duplicates	evidence/latest/scorecard.md	Owner/deadline/status needs the fixture (S02)	☐
Starting state saved	evidence/starting_state/	S04 (synthetic until the fixture arrives)	☐
Negotiated scope and final evidence	docs/product-change.md, evidence/final/	S04	☐
Timestamped screenshots	evidence/screenshots/<timestamp>/	S11	☐
Test report	evidence/latest/test-report.txt	S04	☐
Release check all PASS or explained	docs/release-check.md	S12	☐
API documented and live	docs/api.md, <LIVE_URL>/api/health	S06, S13	☐
PWA installable and offline	e2e/pwa.spec.ts result	S07	☐
Growth experiment: ten real rows	docs/growth-experiment.md	You fill the S11 template	☐
Pilot plan with three metrics	docs/pilot-plan.md	S11	☐
Business case PDF and pitch deck	docs/ (your files)	You provide	☐
README with honest limitations	README.md	S11	☐
Commits only under your name, human-style	GitHub Insights → Contributors; git log	Checked in S12	☐
Demo video (about 3 minutes)	README DEMO_VIDEO_URL	You record and link	☐
Official deadline and demo format confirmed	Organisers	Unknown in the brief	☐